

UNIVERSIDAD NACIONAL DE ASUNCIÓN
FACULTAD POLITÉCNICA

INGENIERÍA EN INFORMÁTICA



**Optimización Escalable y Descentralizada del Tráfico Urbano con
Almacenamiento Verificable para Ciudades Inteligentes**

Kevin Mathias Galeano Saldivar
María José Duarte Kowalewski

Proyecto de Trabajo de Grado presentado en conformidad a los
requisitos para obtener el grado de Ingeniería en Informática
San Lorenzo - Paraguay
Febrero - 2026

UNIVERSIDAD NACIONAL DE ASUNCIÓN
FACULTAD POLITÉCNICA

INGENIERÍA EN INFORMÁTICA



**Optimización Escalable y Descentralizada del Tráfico Urbano con
Almacenamiento Verificable para Ciudades Inteligentes**

**Kevin Mathias Galeano Saldivar
María José Duarte Kowalewski**

San Lorenzo - Paraguay
Febrero - 2026

UNIVERSIDAD NACIONAL DE ASUNCIÓN
FACULTAD POLITÉCNICA

INGENIERÍA EN INFORMÁTICA



**Optimización Escalable y Descentralizada del Tráfico Urbano con
Almacenamiento Verificable para Ciudades Inteligentes**

**Kevin Mathias Galeano Saldivar
María José Duarte Kowalewski**

Asesor:

Cristian Cappelletti

Proyecto de Trabajo de Grado presentado en conformidad a los
requisitos para obtener el grado de Ingeniería en Informática

San Lorenzo - Paraguay

Febrero - 2026

*A nuestras familias,
por su apoyo incondicional y su paciencia
a lo largo de todos estos años.*

AGRADECIMIENTOS

Agradecemos al profesor Marcos Villagra por las discusiones técnicas, sus aportes y el apoyo brindado durante el transcurso de esta investigación.

Agradecemos también al profesor Cristian Cappelletti por su disposición como asesor del trabajo.

Finalmente, y de manera muy especial, agradecemos a nuestras familias por su paciencia, su apoyo incondicional y por estar presentes en cada etapa de este proceso.

Optimización Escalable y Descentralizada del Tráfico Urbano con Almacenamiento Verificable para Ciudades Inteligentes

Autores: Kevin Mathias Galeano Saldivar, María José Duarte Kowalewski

Asesor: Cristian Cappelletti

RESUMEN

Este trabajo presenta el diseño, la implementación y la evaluación experimental de un sistema modular y descentralizado para la optimización adaptativa del control semafórico urbano y la persistencia verificable de los datos asociados. La arquitectura propuesta integra cuatro microservicios: un módulo de simulación de tráfico basado en SUMO (traffic-sim), un módulo de análisis y optimización que combina inferencia difusa tipo Mamdani con optimización por enjambre de partículas (traffic-sync), un módulo de almacenamiento distribuido basado en IPFS y contratos inteligentes sobre una red BlockDAG (traffic-storage), y un módulo de orquestación con interfaz REST (traffic-control). Los módulos se comunican mediante APIs HTTP/JSON, lo que garantiza bajo acoplamiento y facilita la integración con sensores reales.

La evaluación experimental demuestra que el sistema escala de forma aproximadamente lineal con el número de sensores: para 2000 sensores, el ciclo completo de análisis y optimización se completó en 67,6 s ($\approx 0,034$ s por sensor), con consumo de memoria estable entre 198 y 219 MB. En cuanto al almacenamiento verificable, las transacciones alcanzaron confirmación en la red BlockDAG en ≈ 1 s, con un costo inferior a $\$1,2 \times 10^{-5}$ USD por operación, superando en más de cuatro órdenes de magnitud el costo operativo de Ethereum y reduciendo en más del 60 % la latencia de persistencia verificable respecto a plataformas IoT-cloud centralizadas. Los resultados respaldan la viabilidad del sistema para despliegues municipales con recursos computacionales limitados, incluyendo hardware embebido de bajo costo. Los hallazgos principales fueron presentados en la 2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON 2025).

Palabras Clave: optimización de tráfico urbano, control semafórico adaptativo, lógica difusa, optimización por enjambre de partículas, IPFS, BlockDAG, almacenamiento verificable, microservicios, ciudad inteligente.

Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments

Authors: Kevin Mathias Galeano Saldivar, María José Duarte Kowalewski

Advisor: Cristian Cappelletti

ABSTRACT

This work presents the design, implementation, and experimental evaluation of a modular, decentralized system for adaptive urban traffic signal optimization and verifiable data persistence. The proposed architecture integrates four microservices: a SUMO-based traffic simulation module (*traffic-sim*), an analysis and optimization module combining Mamdani fuzzy inference with Particle Swarm Optimization (*traffic-sync*), a distributed storage module built on IPFS and smart contracts over a BlockDAG network (*traffic-storage*), and an orchestration module with a REST interface (*traffic-control*). All modules communicate via HTTP/JSON APIs, ensuring loose coupling and straightforward integration with real-world sensor infrastructure.

Experimental evaluation demonstrates approximately linear scalability with the number of sensors: for 2000 sensors, the full analysis-and-optimization cycle completed in 67.6 s (≈ 0.034 s per sensor), with stable memory consumption between 198 and 219 MB. For verifiable storage, transactions reached confirmation on the BlockDAG network in ≈ 1 s at a cost below $\$1.2 \times 10^{-5}$ USD per operation, outperforming Ethereum by more than four orders of magnitude in cost and reducing verifiable-persistence latency by over 60 % compared to centralized IoT-cloud platforms. These results support the system's viability for municipal deployments with limited computational resources, including low-cost embedded hardware. The main findings were presented at the *2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies* (CHILECON 2025).

Keywords: urban traffic optimization, adaptive signal control, fuzzy logic, particle swarm optimization, IPFS, BlockDAG, verifiable storage, microservices, smart city.

ÍNDICE

Página

1. INTRODUCCIÓN	1
1.1. Motivación y contexto	2
1.2. Objetivos	2
1.2.1. Objetivo general	2
1.2.2. Objetivos específicos	2
1.3. Contribuciones	3
1.4. Estructura del documento	4
2. INGENIERÍA DE TRÁFICO Y CONTROL SEMAFÓRICO	5
2.1. Semáforos para el Control del Tránsito de Vehículos	5
2.2. Semáforos de Tiempos Fijos o Predeterminados	6
2.3. Semáforos Accionados por el Tránsito	7
2.4. Técnicas Algorítmicas y Computacionales para Control Adaptativo	8
2.5. Parámetros Clásicos de Diseño de Tiempos Semafóricos	10
2.6. Modelado del Control Semafórico como Problema de Optimización	11
2.7. Coordinación de Semáforos y Progresión de Pelotones	12
2.8. Consideraciones Operativas y Métricas de Desempeño	13
2.9. Integración de Técnicas de Optimización y Control Difuso	14
3. VISIÓN GENERAL DEL SISTEMA	16
3.1. Módulos Principales	16
3.2. Flujo General de Información	17
3.3. Justificación de la Arquitectura	17
4. SIMULACIÓN DE TRÁFICO	20
4.1. Arquitectura del Módulo	20
4.2. Flujo de Integración	22
4.3. Escenario de Simulación y Parámetros de Detección	22
5. ALMACENAMIENTO DISTRIBUIDO Y TECNOLOGÍAS DESCENTRALIZADAS	24
5.1. Blockchain	24
5.2. BlockDAG	25
5.3. IPFS	26
5.4. Interfaz del Contrato TrafficStorage	28
6. OPTIMIZACIÓN Y SINCRONIZACIÓN DE TRÁFICO	31
6.1. Lógica Difusa Mamdani	31
6.2. Optimización de Enjambre de Partículas (PSO)	34

6.3. Ejemplos de Ciclos de Optimización	37
7. CONTROL Y ORQUESTACIÓN DEL SISTEMA	39
7.1. Arquitectura del Módulo	39
7.2. Flujo de Procesamiento	40
7.3. Interacción con los Módulos	42
7.4. Contrato de la API REST	44
8. EVALUACIÓN EXPERIMENTAL DEL SISTEMA	45
8.1. Evaluación del módulo traffic-sync	46
8.1.1. Entorno y Protocolo de Prueba	46
8.1.2. Tiempo de Ejecución	47
8.1.3. Uso de CPU y Memoria	48
8.2. Evaluación del módulo traffic-storage	49
8.2.1. Métricas de Transacción Base	49
8.2.2. Pruebas con Escenarios de Tráfico Vehicular	50
8.2.3. Descomposición del Pipeline de Almacenamiento	51
8.2.4. Verificación de Integridad y Trazabilidad	52
8.2.5. Consumo de Gas y Costos Operativos	53
8.3. Comparación con Tecnologías Alternativas	53
8.3.1. BlockDAG frente a Ethereum	54
8.3.2. Reducción de Latencia respecto a Plataformas Centralizadas	54
8.4. Discusión	54
8.4.1. Viabilidad del Sistema	55
8.4.2. Desafíos Identificados y Limitaciones	55
9. CONCLUSIÓN	57
9.1. Resultados principales	57
9.2. Divulgación científica	58
9.3. Limitaciones	58
9.4. Trabajo futuro	59
9.5. Síntesis final	60
REFERENCIAS	61

LISTA DE FIGURAS

Página

3.1. Arquitectura general del sistema mostrando los cuatro módulos principales y el flujo de información.	19
4.1. Arquitectura del módulo traffic-sim y sus componentes principales [38].	21
5.1. Estructura básica de una cadena de bloques, mostrando la dependencia criptográfica entre bloques consecutivos [39].	25
5.2. Arquitectura básica de IPFS: fragmentación de archivos, generación de CID y recuperación distribuida [8].	27
5.3. Diagrama de secuencia del flujo de almacenamiento distribuido en traffic-storage.	28
6.1. Variable lingüística <i>velocidad promedio</i> con tres conjuntos difusos modelados mediante funciones de membresía trapezoidal, triangular y aproximadamente gaussiana [27].	32
6.2. Etapas principales de un sistema de inferencia difusa tipo Mamdani aplicado a congestión vehicular [25, 48].	33
6.3. Esquema conceptual del enjambre PSO en un espacio de búsqueda bidimensional [7].	35
6.4. Arquitectura del módulo traffic-sync integrando lógica difusa Mamdani y optimización PSO.	37
7.1. Arquitectura del módulo traffic-control mostrando el flujo de orquestación. El almacenamiento principal se realiza mediante IPFS y BlockDAG a través de traffic-storage, mientras que la base de datos SQL se utiliza únicamente como índice auxiliar de metadatos.	41
7.2. Diagrama de secuencia del flujo completo de procesamiento en el sistema distribuido.	43
8.1. Tiempo de ejecución del módulo traffic-sync en función de la cantidad de sensores simulados. El crecimiento aproximadamente lineal ($\approx 0,034$ s/sensor) confirma la escalabilidad del módulo.	48
8.2. Uso promedio de CPU y consumo de memoria RAM del módulo traffic-sync bajo diferentes cargas de sensores simulados. El CPU disminuye conforme aumenta la concurrencia; la memoria permanece prácticamente estable en el rango 200–220 MB.	49
8.3. Tiempos promedio de subida y descarga del módulo traffic-storage por escenario de tráfico. Las barras de error representan ± 1 desviación estándar. El incremento de latencia de subida entre el escenario bajo y el alto es del 8,4 %.	51

8.4. Desglose del tiempo de ejecución en las operaciones de subida y descarga del módulo traffic-storage. La confirmación en la red BlockDAG domina la latencia de subida ($\approx 50\%$), mientras que la descarga se completa en $\approx 1,6$ s sin necesidad de esperar confirmación.	52
--	----

LISTA DE TABLAS

Página

4.1. Esquema del payload enviado de traffic-sim a traffic-sync ante un cuello de botella.	23
5.1. Comparación entre Blockchain Tradicional y BlockDAG	26
5.2. Interfaz pública del contrato <code>TrafficStorage.sol</code>	29
6.1. Parámetros de las funciones de membresía del sistema difuso Mamdani en traffic-sync	32
6.2. Base de reglas completa del sistema de inferencia difusa Mamdani (27 reglas) . .	34
6.3. Parámetros de configuración del algoritmo PSO en traffic-sync	36
6.4. Resultados reales del módulo traffic-sync ante cuatro escenarios de tráfico. . . .	38
7.1. Endpoints de la API REST de traffic-control.	44
8.1. Especificaciones del entorno experimental	47
8.2. Métricas de rendimiento del módulo traffic-storage (IPFS + BlockDAG)	50
8.3. Tiempos de subida y descarga por escenario de tráfico (5 corridas válidas por escenario)	51
8.4. Muestra de transacciones exitosas registradas durante las pruebas	52
8.5. Consumo de gas y costo estimado por tipo de operación de almacenamiento . . .	53
8.6. Comparación técnica entre Ethereum y BlockDAG en el contexto de monitoreo urbano de tráfico. Valores de BlockDAG medidos en Primordial TestNet.	54

LISTA DE ACRÓNIMOS Y SÍMBOLOS

Infraestructura y arquitectura de software

API	<i>Application Programming Interface</i> ; interfaz de programación que expone funcionalidades de un servicio para ser consumidas por otros módulos o clientes externos.
REST	<i>Representational State Transfer</i> ; estilo arquitectónico para el diseño de APIs HTTP basado en recursos y verbos estándar.
HTTP /	<i>HyperText Transfer Protocol (Secure)</i> ; protocolos de transporte
HTTPS	sobre los que se exponen las APIs del sistema.
JSON	<i>JavaScript Object Notation</i> ; formato de intercambio de datos estructurados utilizado en los mensajes entre módulos.
URL	<i>Uniform Resource Locator</i> ; identificador de recursos web utilizado para apuntar a endpoints y nodos del sistema.
RPC	<i>Remote Procedure Call</i> ; modelo de invocación remota empleado en la comunicación con nodos de la red BlockDAG.
SQL	<i>Structured Query Language</i> ; lenguaje de consulta utilizado en la base de datos relacional de metadatos operativos.
CPU	<i>Central Processing Unit</i> ; procesador principal del host, recurso monitoreado en los experimentos de rendimiento.
RAM	<i>Random Access Memory</i> ; memoria principal del host, recurso monitoreado en los experimentos de escalabilidad.
IoT	<i>Internet of Things</i> ; paradigma de dispositivos interconectados; fuente de datos real a integrar en despliegues futuros.
FastAPI	<i>FastAPI</i> ; framework web asíncrono de Python utilizado para implementar las APIs REST de los módulos del sistema.

Simulación y tráfico urbano

SUMO	<i>Simulation of Urban MObility</i> ; plataforma de simulación de tráfico urbano de código abierto utilizada en el módulo <i>traffic-sim</i> .
------	--

TraCI	<i>Traffic Control Interface</i> ; protocolo de comunicación de SUMO que permite controlar y consultar la simulación en tiempo real.
ITS	<i>Intelligent Transportation Systems</i> ; sistemas inteligentes de transporte; marco general en el que se inscribe este trabajo.
MILP	<i>Mixed-Integer Linear Programming</i> ; programación entera mixta; formulación clásica del problema de optimización de planes semafóricos.
TPS	Transacciones por segundo; métrica de rendimiento empleada para comparar el throughput de BlockDAG frente a blockchain lineal.

Inteligencia computacional

PSO	<i>Particle Swarm Optimization</i> ; algoritmo de optimización por enjambre de partículas utilizado para ajustar los tiempos semafóricos.
FIS	<i>Fuzzy Inference System</i> ; sistema de inferencia difusa; implementado con modelo Mamdani para clasificar el nivel de congestión.

Almacenamiento descentralizado

IPFS	<i>InterPlanetary File System</i> ; sistema de almacenamiento distribuido por contenido utilizado para persistir los datos de tráfico.
CID	<i>Content Identifier</i> ; identificador criptográfico de contenido en IPFS; permite verificar la integridad de los datos almacenados.
DAG	<i>Directed Acyclic Graph</i> ; grafo dirigido acíclico; estructura de datos base de IPFS y de las redes BlockDAG.
BlockDAG	Red de bloques organizada como grafo dirigido acíclico en lugar de cadena lineal; permite mayor throughput y confirmación rápida de transacciones.
PHANTOM /	Protocolos de consenso y ordenamiento para redes BlockDAG que definen
GHOSTDAG	la cadena principal y resuelven conflictos entre bloques paralelos.
PoW	<i>Proof of Work</i> ; mecanismo de consenso basado en cómputo intensivo; referencia comparativa frente a BlockDAG.
PoS	<i>Proof of Stake</i> ; mecanismo de consenso basado en participación económica; alternativa energéticamente eficiente a PoW.

P2P	<i>Peer-to-Peer</i> ; modelo de red entre pares utilizado por IPFS y las redes BlockDAG para distribuir contenido y bloques.
DHT	<i>Distributed Hash Table</i> ; tabla de hash distribuida que usa IPFS para localizar nodos que almacenan un CID dado.
Solidity	Lenguaje de programación de contratos inteligentes utilizado para implementar el registro inmutable en la red BlockDAG.
SHA	<i>Secure Hash Algorithm</i> ; familia de funciones de resumen criptográfico empleadas en la construcción de CIDs e identificadores de bloque.

Símbolos matemáticos

x_i	Posición de la partícula i en el espacio de búsqueda del PSO; representa una configuración candidata de tiempos semafóricos.
v_i	Velocidad de la partícula i en el PSO; determina la magnitud y dirección del desplazamiento en cada iteración.
p_i	Mejor posición histórica (<i>personal best</i>) de la partícula i .
g	Mejor posición global (<i>global best</i>) encontrada por el enjambre.
w	Factor de inercia del PSO; controla el peso de la velocidad previa en la actualización de la trayectoria.
c_1, c_2	Coefficientes de aceleración cognitivo y social del PSO.
r_1, r_2	Números aleatorios uniformes en $[0, 1]$ usados en la ecuación de actualización de velocidad del PSO.
C	Longitud de ciclo semafórico; duración total de una repetición completa de las fases.
g_k	Tiempo de verde asignado a la fase k dentro del ciclo C .
μ	Grado de pertenencia en lógica difusa; valor en $[0, 1]$ que indica cuánto pertenece un valor a un conjunto difuso.
ϕ_j	Función de pertenencia j ; define la forma del conjunto difuso (triangular, trapezoidal, gaussiana).

CAPÍTULO 1

INTRODUCCIÓN

La congestión del tráfico urbano representa uno de los problemas más persistentes y costosos de las ciudades contemporáneas. El crecimiento sostenido del parque automotor, combinado con una infraestructura vial que raramente crece al mismo ritmo, genera demoras acumuladas que impactan directamente en la productividad económica, la calidad del aire y la calidad de vida de los habitantes [1, 2]. En ciudades de tamaño medio de América Latina, donde los recursos para grandes obras de infraestructura son escasos, la optimización del control semafórico existente se presenta como una de las intervenciones de mayor relación costo-beneficio para mejorar la fluidez del tráfico sin requerir expansión física de la red vial [3].

Los sistemas de control semafórico tradicionales basados en tiempos fijos o predeterminados presentan limitaciones fundamentales frente a la variabilidad inherente de la demanda urbana. Un plan de señales diseñado para el pico de la mañana es ineficiente en el valle del mediodía; un plan calibrado para días laborables falla en eventos o festividades. Esta rigidez operativa motiva el desarrollo de estrategias adaptativas capaces de ajustar los parámetros de señalización (duración de ciclo, distribución de tiempos verdes, desplazamientos entre intersecciones) en respuesta a las condiciones observadas en tiempo real [1, 3, 4].

La convergencia de tres tendencias tecnológicas abre nuevas posibilidades para abordar este problema. En primer lugar, la disponibilidad masiva de sensores de bajo costo y plataformas de simulación de tráfico de código abierto, como SUMO (Simulation of Urban MObility), permite evaluar y validar estrategias de control bajo condiciones controladas y reproducibles antes de su despliegue en campo [5]. En segundo lugar, las técnicas de inteligencia computacional, en particular la lógica difusa y los algoritmos de optimización metaheurística, ofrecen mecanismos para modelar el conocimiento experto sobre congestión y para buscar configuraciones semaforicas óptimas en espacios de decisión complejos, sin requerir modelos analíticos exactos de la demanda [6, 7]. En tercer lugar, las tecnologías de almacenamiento descentralizado (IPFS y

redes BlockDAG) permiten registrar los datos generados por el sistema de forma inmutable y auditable, sin depender de un operador central de confianza, lo que resulta especialmente relevante para infraestructuras de gestión pública donde la trazabilidad y la rendición de cuentas son requisitos institucionales [8, 9].

1.1. Motivación y contexto

En el contexto paraguayo, la gestión del tráfico urbano en ciudades intermedias y en el área metropolitana de Asunción enfrenta el desafío adicional de operar con presupuestos municipales limitados y con infraestructura tecnológica heterogénea. Las soluciones comerciales de control adaptativo, ampliamente desplegadas en ciudades de países desarrollados, suelen implicar costos de licenciamiento, dependencia de proveedores y dificultades de integración con sistemas legados, barreras que frenan su adopción a escala local. Este escenario plantea la necesidad de explorar alternativas basadas en software libre, componentes de hardware de consumo y arquitecturas abiertas que puedan desplegarse y mantenerse con los recursos técnicos disponibles en municipios de la región.

Paralelamente, la creciente adopción de paradigmas de smart city en la planificación urbana de América Latina genera demanda de soluciones que no solo optimicen parámetros operativos, sino que también garanticen la integridad y disponibilidad de los datos generados. En sistemas de gestión pública, la capacidad de auditar retrospectivamente el comportamiento del sistema (verificar qué configuraciones se aplicaron, cuándo y con qué efecto) constituye un requisito de transparencia que las plataformas centralizadas tradicionales satisfacen de forma insuficiente.

1.2. Objetivos

1.2.1. Objetivo general

Diseñar, implementar y evaluar experimentalmente un sistema modular y descentralizado para la optimización adaptativa del control semafórico urbano y la persistencia verificable de los datos asociados, utilizando técnicas de inteligencia computacional y tecnologías de almacenamiento distribuido sobre hardware de consumo estándar.

1.2.2. Objetivos específicos

- Desarrollar un módulo de simulación de tráfico que genere escenarios controlados y reproducibles mediante SUMO, incluyendo detección automática de cuellos de botella y exposición de métricas agregadas por intersección semafórica.
- Diseñar e implementar un módulo de análisis y optimización que clasifique el estado de congestión mediante inferencia difusa tipo Mamdani y ajuste los tiempos semafóricos mediante optimización por enjambre de partículas (PSO).

- Construir un módulo de persistencia verificable que almacene datos de tráfico y resultados de optimización en IPFS, registrando los identificadores de contenido en una red BlockDAG mediante contratos inteligentes para garantizar inmutabilidad y trazabilidad.
- Integrar los módulos anteriores mediante un servicio de orquestación con interfaz REST que coordine el flujo completo desde la recepción de observaciones hasta la confirmación del almacenamiento distribuido.
- Evaluar experimentalmente el desempeño del sistema en términos de escalabilidad, consumo de recursos, latencia de almacenamiento y costo operativo, comparando con alternativas basadas en Ethereum y plataformas centralizadas IoT-cloud.

1.3. Contribuciones

El presente trabajo realiza las siguientes contribuciones:

1. Una arquitectura de microservicios de código abierto que integra simulación de tráfico, optimización difusa-PSO y almacenamiento descentralizado en un flujo operativo unificado, demostrada en hardware de consumo estándar.
2. Un protocolo de persistencia verificable para datos de tráfico urbano basado en IPFS y contratos inteligentes sobre BlockDAG, con confirmación en ≈ 1 s y costo inferior a $\$1,2 \times 10^{-5}$ USD por transacción, superando en más de cuatro órdenes de magnitud el costo operativo de Ethereum.
3. Caracterización empírica del perfil de escalabilidad del pipeline difuso-PSO, con crecimiento aproximadamente lineal en tiempo de procesamiento ($\approx 0,034$ s/sensor), estabilidad de memoria entre 198 y 219 MB y decremento de uso de CPU conforme aumenta la concurrencia, estableciendo una línea base reproducible para comparaciones futuras con alternativas de control adaptativo en hardware de consumo estándar.
4. Un conjunto de experimentos reproducibles, con configuraciones, datos y métricas documentados, que sirven de línea base para investigaciones futuras en optimización de tráfico con requisitos de trazabilidad verificable.

Los hallazgos principales fueron publicados en el artículo *Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments*, presentado en la 2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON 2025), Valparaíso, Chile [10].

1.4. Estructura del documento

El resto del documento se organiza de la siguiente manera. El Capítulo 2 presenta el marco teórico que fundamenta el trabajo: ingeniería de tráfico y control semafórico, lógica difusa y sistemas de inferencia Mamdani, optimización por enjambre de partículas, IPFS como sistema de almacenamiento distribuido por contenido, y las estructuras BlockDAG y blockchain empleadas para el registro inmutable. El Capítulo 3 describe la visión general del sistema, su arquitectura modular y el flujo de información entre componentes. Los Capítulos 4 a 7 detallan el diseño e implementación de cada módulo: simulación traffic-sim, optimización traffic-sync, almacenamiento verificable traffic-storage y orquestación traffic-control. El Capítulo 8 presenta la evaluación experimental con los resultados de escalabilidad, latencia y costo. Finalmente, el Capítulo 9 recoge las conclusiones, limitaciones y líneas de trabajo futuro.

CAPÍTULO 2

INGENIERÍA DE TRÁFICO Y CONTROL SEMAFÓRICO

La ingeniería de tráfico es la rama de la ingeniería del transporte que se ocupa del planeamiento, diseño, operación y gestión de instalaciones viales, con el objetivo de proporcionar un movimiento seguro y eficiente de personas y mercancías [2, 11]. Su enfoque combina el análisis de la interacción entre usuarios, vehículos e infraestructura con el estudio de parámetros macroscópicos de flujo, como volumen, velocidad y densidad, para evaluar y mejorar el desempeño de redes viales urbanas y periurbanas.

Dentro de este campo, el control semafórico constituye una de las herramientas centrales para regular el derecho de paso en intersecciones donde confluyen flujos vehiculares y peatonales conflictivos [2, 12]. El diseño de planes de señalización semafórica abarca la definición de fases, la asignación de tiempos de verde, amarillo y rojo, y la coordinación entre intersecciones vecinas, con el fin de equilibrar seguridad, capacidad y niveles de servicio. La evolución desde esquemas de tiempos fijos hacia controles actuados y adaptativos refleja la necesidad de responder a patrones de demanda cada vez más variables, especialmente en entornos urbanos complejos.

2.1. Semáforos para el Control del Tránsito de Vehículos

Los semáforos constituyen el principal dispositivo de control del derecho de paso en intersecciones urbanas y periurbanas con conflictos significativos de circulación vehicular y peatonal [13]. En el Manual de Carreteras del Paraguay se establece que los semáforos para el control del tránsito de vehículos deben emplearse para asignar tiempos de paso y de detención a los distintos movimientos que confluyen en la intersección, con el objetivo de reducir conflictos, ordenar las maniobras y aumentar la seguridad vial [13]. Esta función se complementa con el control de flujos peatonales y, cuando corresponde, con fases específicas para giros, carriles

exclusivos y prioridades especiales (por ejemplo, transporte público o vehículos de emergencia) [13].

Desde el punto de vista de la ingeniería de tráfico, un semáforo vehicular puede interpretarse como un sistema de control discreto en el tiempo que alterna estados de indicación verde, amarilla y roja para cada grupo de movimientos, de forma tal que en ningún instante se autoricen simultáneamente movimientos incompatibles [13, 2]. El diseño del plan de fases considera la geometría de la intersección, los volúmenes de tránsito por aproximación, la presencia de giros a la izquierda y a la derecha, los flujos peatonales y, en su caso, la coordinación con semáforos próximos en el corredor [13, 11]. Las indicaciones luminosas verde, amarilla y roja poseen significado normado asociado a las obligaciones legales de detenerse o avanzar, y se complementan con flechas direccionales cuando es necesario discriminar maniobras permitidas dentro de una misma aproximación [13].

En la normativa paraguaya se especifican además criterios de ubicación, número de caras y área de influencia de cada cabezal semafórico, de modo que las indicaciones resulten claramente visibles para todos los usuarios a los que se dirigen [13]. Estos lineamientos aseguran que el dispositivo de control implementado por el sistema desarrollado en el presente trabajo sea compatible con la infraestructura semafórica existente y respetuoso de las disposiciones vigentes en el país.

2.2. Semáforos de Tiempos Fijos o Predeterminados

Los semáforos de tiempos fijos o predeterminados operan con un plan de señales cuya secuencia de fases y tiempos se define previamente y se repite cíclicamente, sin interacción directa con el tránsito en tiempo real [13]. El Manual de Carreteras del Paraguay indica que estos equipos asignan a cada fase un tiempo de verde y un tiempo intermedio (amarillo y, si corresponde, tiempo de despeje en rojo) calculados a partir de volúmenes de tránsito de diseño, análisis de capacidad y niveles de servicio objetivo [13]. Los tiempos de fase se mantienen constantes durante el período de operación del plan, aunque se contempla el uso de distintos planes fijos según la franja horaria (pico de la mañana, valle, pico de la tarde, periodo nocturno) [13, 2].

La principal ventaja de los semáforos de tiempos fijos reside en su simplicidad de diseño, facilidad de implementación y previsibilidad del patrón de señales, lo que se traduce en menores requerimientos de equipamiento (no se requieren detectores de vehículos) y en un mantenimiento relativamente sencillo [2, 11]. Sin embargo, este tipo de control presenta limitaciones en entornos con variaciones significativas de la demanda, como redes urbanas con fuertes diferencias entre horas pico y valle o con eventos recurrentes que alteran los patrones de flujo, ya que el plan fijo no se adapta a la ocupación real de las aproximaciones y puede generar tiempos de verde desaprovechados o demoras excesivas en ciertas corrientes [1, 3].

Desde la perspectiva de la optimización, el diseño de planes de tiempos fijos puede formularse como un problema de programación entera mixta (Mixed-Integer Linear Programming,

MILP) en el que las variables de decisión, a saber, la duración del ciclo, los tiempos de verde por fase (splits) y, en el caso de redes coordinadas, los desplazamientos temporales entre intersecciones (offsets), son esencialmente discretas o deben satisfacer relaciones de discretización [14, 15]. En este marco, la función objetivo típicamente busca minimizar el retraso total de la red o maximizar la banda de progresión de pelotones a lo largo de un corredor, sujeta a restricciones de consistencia de ciclo, tiempos mínimos de verde, capacidad y sincronización [14, 16]. Los métodos analíticos clásicos, como las fórmulas de Webster [17], constituyen soluciones cerradas para versiones simplificadas de este problema: asumen una intersección aislada, demanda constante y condiciones de estado estacionario, permitiendo obtener expresiones analíticas para el ciclo óptimo y los splits que minimizan el retraso medio. Sin embargo, estas soluciones ignoran las interdependencias entre intersecciones, la naturaleza estocástica de los arribos y la presencia de restricciones operativas complejas [15, 1].

Cuando se considera la optimización de redes semafóricas con múltiples intersecciones coordinadas, el problema adquiere dimensiones combinatorias elevadas: el número de configuraciones posibles crece exponencialmente con el número de nodos, fases y rangos de valores admisibles para ciclos y offsets, lo que convierte al problema en NP-difícil [14, 15]. En estos casos, los métodos exactos de programación entera mixta pueden resultar computacionalmente intratables para redes de tamaño medio o grande, particularmente cuando se requiere resolver el problema en tiempo razonable para permitir ajustes frecuentes de los planes en respuesta a cambios de demanda. Por ello, la literatura ha recurrido a técnicas heurísticas y metaheurísticas, como algoritmos genéticos, simulated annealing y búsqueda local, para encontrar soluciones de buena calidad sin garantizar la optimalidad global [18, 19]. En particular, los algoritmos genéticos han mostrado eficacia para optimizar planes de tiempos fijos en condiciones de demanda sobresaturada, donde los modelos analíticos tradicionales pierden validez y se requiere exploración robusta del espacio de soluciones [18].

Diversos estudios en ingeniería de tráfico han mostrado que, en condiciones de demanda altamente variable, los esquemas de control fijo tienden a producir mayores tiempos de retraso medio, un mayor número de paradas y colas más extensas que los sistemas actuados o adaptativos [1, 3]. Este comportamiento motiva la búsqueda de estrategias de control más flexibles, como las basadas en lógica difusa o algoritmos de optimización metaheurística, que constituyen el foco del trabajo.

2.3. Semáforos Accionados por el Tránsito

Los semáforos accionados por el tránsito utilizan información en tiempo real captada por detectores (por ejemplo, lazos inductivos, sensores de radar o cámaras de video) ubicados en las aproximaciones para modificar la duración de las fases dentro de límites mínimos y máximos definidos [13]. El Manual de Carreteras del Paraguay distingue entre control parcialmente actuado (al menos una aproximación bajo control por detectores) y totalmente actuado (todas las aproximaciones controladas por demanda), estableciendo criterios de diseño para la ubicación

de detectores, la distancia respecto de la línea de detención y los períodos iniciales mínimos y extensiones de verde en función de la velocidad que comprende el 85 % del tránsito en el acceso [13].

En estos sistemas, el controlador decide en cada ciclo si extiende o termina el verde de una fase atendiendo a la detección de vehículos, lo que permite reducir tiempos de verde desaprovechados en movimientos con baja demanda y asignar capacidad adicional a las aproximaciones más cargadas [11, 1]. La literatura sobre control semafórico actuado y adaptativo destaca que este tipo de sistemas mejora el desempeño frente a planes fijos, especialmente en condiciones de demanda fluctuante, al disminuir el retraso medio por vehículo y el número de paradas, y al reducir la probabilidad de colas críticas en aproximaciones dominantes [1, 3].

El uso de detectores también permite implementar estrategias avanzadas como extensiones de verde para “limpiar” colas residuales, prioridad al transporte público o prioridad a vehículos de emergencia, resultando particularmente relevante en corredores urbanos con alta variabilidad de flujos [1, 20]. Estos conceptos constituyen la base sobre la cual se articula el sistema de control inteligente propuesto en este trabajo, donde un controlador difuso tipo Mamdani y algoritmos de optimización como Particle Swarm Optimization (PSO) se integran para ajustar dinámicamente parámetros de operación semafórica en respuesta a condiciones de tráfico cambiantes; el enfoque híbrido resultante se describe en detalle en la Sección 2.9.

2.4. Técnicas Algorítmicas y Computacionales para Control Adaptativo

La creciente disponibilidad de sensores de tráfico en tiempo real, el incremento de la capacidad de procesamiento computacional y los avances en inteligencia artificial han posibilitado el desarrollo de estrategias de control semafórico que van más allá de los enfoques clásicos basados en planes de tiempo fijo o en lógicas actuadas simples [1, 3]. Estas técnicas algorítmicas y computacionales permiten abordar el problema de optimización de tiempos semafóricos en condiciones de demanda estocástica y no estacionaria, superando las limitaciones de los métodos analíticos tradicionales que asumen flujos uniformes y equilibrio de estado [1, 4].

Entre las técnicas más consolidadas se encuentran los sistemas de control adaptativo en tiempo real, como SCOOT (Split, Cycle and Offset Optimization Technique) y SCATS (Sydney Coordinated Adaptive Traffic System), que ajustan automáticamente los parámetros de señalización a partir de información de ocupación y flujo captada por detectores ubicados aguas arriba de las intersecciones [21, 22]. SCOOT, desarrollado en el Reino Unido, emplea modelos de tráfico microscópicos para predecir la llegada de pelotones y ajusta ciclo, splits y offsets de forma continua, minimizando una función de costo basada en demoras y paradas [21, 1]. SCATS, implementado originalmente en Australia, utiliza un enfoque de biblioteca de planes (plan library) en el que un controlador central selecciona dinámicamente el plan más apropiado de un conjunto predefinido en función de las condiciones de tráfico observadas, y

ajusta los tiempos de verde dentro de límites establecidos [22, 20]. Ambos sistemas han demostrado reducciones significativas en demoras y emisiones en redes urbanas de mediana y gran escala, aunque requieren infraestructura de detección extensa y calibración cuidadosa de sus parámetros operativos.

Paralelamente, el campo de la optimización metaheurística ha aportado algoritmos capaces de explorar eficientemente el espacio de soluciones de problemas combinatorios complejos, evitando quedar atrapados en óptimos locales. Los algoritmos genéticos (Genetic Algorithms, GA) emulan procesos de evolución biológica (selección, cruce y mutación) para hacer evolucionar poblaciones de soluciones candidatas (configuraciones de ciclo, splits y offsets) hacia regiones de alta calidad del espacio de búsqueda, y han sido aplicados con éxito a la optimización de redes semafóricas coordinadas y a la optimización multiobjetivo que balancea demoras, emisiones y equidad [19, 1]. Particle Swarm Optimization (PSO), inspirado en el comportamiento social de bandadas de aves o cardúmenes de peces, representa cada solución candidata como una “partícula” que se desplaza en el espacio de parámetros guiada por su propia experiencia (mejor posición histórica individual) y por la experiencia del conjunto (mejor posición global del enjambre), logrando convergencia rápida y robusta en problemas de optimización de tiempos semafóricos [7, 23, 24]. Otros algoritmos metaheurísticos aplicados al dominio incluyen *simulated annealing*, búsqueda tabú y optimización por colonias de hormigas, cada uno con fortalezas específicas en términos de diversidad de exploración, velocidad de convergencia y manejo de múltiples objetivos [1].

En el ámbito de la inteligencia artificial, los sistemas basados en lógica difusa (fuzzy logic) permiten incorporar conocimiento experto y razonamiento aproximado en el control de semáforos, modelando conceptos lingüísticos como “congestión alta”, “demanda moderada” o “cola larga” mediante funciones de pertenencia y reglas de inferencia del tipo “si-entonces” [25, 26]. Los controladores difusos tipo Mamdani mapean variables de entrada continuas (por ejemplo, volumen, velocidad, densidad) a decisiones de control (extensión de verde, cambio de fase) de forma intuitiva y transparente, facilitando la interpretación y el ajuste de los parámetros por parte de operadores humanos [27, 28]. Las redes neuronales artificiales (Artificial Neural Networks, ANN) han sido exploradas como aproximadores universales de funciones complejas en el contexto de predicción de tráfico y control adaptativo, aunque su naturaleza de “caja negra” dificulta la interpretabilidad de las decisiones [1]. Más recientemente, las técnicas de aprendizaje por refuerzo (Reinforcement Learning, RL), en particular Q-learning y Deep Q-Networks (DQN), han ganado prominencia al permitir que un agente de control aprenda políticas óptimas de asignación de verde a partir de la interacción directa con el entorno de tráfico, sin requerir un modelo explícito del sistema y adaptándose automáticamente a patrones de demanda cambiantes [29, 4]. Estos enfoques de aprendizaje profundo han mostrado resultados prometedores en simulaciones, superando en algunos casos a SCOOT y SCATS en escenarios de alta variabilidad, aunque aún enfrentan desafíos de estabilidad, seguridad y aceptación regulatoria para su despliegue en sistemas reales [4].

Finalmente, enfoques más radicales como los sistemas autoorganizados proponen que cada

intersección opere de forma autónoma, coordinándose con sus vecinas mediante intercambio de información local y reglas de decisión descentralizadas, sin depender de un controlador central [30]. Esta perspectiva se inspira en fenómenos naturales de autoorganización y busca mayor robustez frente a fallos y escalabilidad en redes extensas, aunque plantea interrogantes sobre garantías de desempeño y estabilidad global del sistema.

En conjunto, estas técnicas algorítmicas y computacionales representan una evolución desde los paradigmas tradicionales de control fijo y actuado hacia estrategias verdaderamente adaptativas, capaces de responder de forma dinámica y autónoma a la variabilidad inherente del tráfico urbano moderno. El presente trabajo se inscribe en esta línea, empleando lógica difusa tipo Mamdani combinada con PSO; la descripción detallada del enfoque híbrido adoptado se presenta en la Sección 2.9.

2.5. Parámetros Clásicos de Diseño de Tiempos Semafóricos

En el diseño clásico de control semafórico, la configuración de una intersección se describe mediante un conjunto reducido de parámetros numéricos que determinan su desempeño operativo: la longitud de ciclo, la repartición de verdes por fase (splits), los tiempos de amarillo y todo-rojo, y los tiempos perdidos en cada etapa del ciclo [2, 11]. La longitud de ciclo C representa la duración total de una repetición completa de las fases del semáforo, mientras que los splits determinan qué fracción de ese ciclo se asigna al verde efectivo de cada movimiento o grupo de movimientos. Los tiempos de amarillo y todo-rojo se definen para advertir el fin del derecho de paso y garantizar el despeje seguro de la intersección antes de otorgar el verde a flujos conflictivos, y los tiempos perdidos agrupan intervalos en los que, por razones de seguridad o arranque, la capacidad efectiva de la intersección es menor a la teórica [2].

Estos parámetros influyen directamente en la capacidad y el retraso en cada aproximación. Para una misma demanda, ciclos más largos tienden a aumentar la capacidad por fase pero también pueden incrementar el tiempo medio de espera si los usuarios deben esperar largos intervalos de rojo, mientras que ciclos muy cortos reducen el retraso pero pueden volverse ineficientes si el tiempo perdido ocupa una proporción significativa del ciclo [11, 1]. La capacidad se relaciona con la proporción de verde asignada a cada movimiento y con la saturación de los carriles, en tanto que el retraso medio por vehículo, el número de paradas y la longitud de colas son métricas fundamentales para evaluar la calidad de servicio de un plan de tiempos [2, 12].

Métodos de diseño tradicional, como la formulación de Webster, proporcionan expresiones para estimar un ciclo “óptimo” fijo a partir de los volúmenes de diseño y de los tiempos perdidos, buscando un compromiso entre capacidad y retraso medio [17, 12]. En la práctica, estos valores se calculan para condiciones de flujo representativas y se implementan como planes de tiempo fijo, eventualmente diferenciados por franja horaria. En contraste, el enfoque desarrollado en este trabajo utiliza un sistema de inferencia difusa tipo Mamdani y un algoritmo de optimización

PSO para ajustar dinámicamente la longitud de ciclo efectiva, los splits y otros parámetros de operación en función de mediciones de tráfico en tiempo real, con el objetivo de reducir retrasos y mejorar el desempeño frente a condiciones de demanda variables [1, 3].

2.6. Modelado del Control Semafórico como Problema de Optimización

El diseño y operación de sistemas de control semafórico puede formularse como un problema de optimización combinatoria en el que se busca determinar los valores de un conjunto de variables de decisión que minimicen (o maximicen) una función objetivo, respetando un conjunto de restricciones operativas y de seguridad [15, 16]. Esta perspectiva permite trascender los enfoques tradicionales basados en fórmulas empíricas y abordar de manera sistemática la complejidad inherente a redes urbanas con demanda variable y múltiples intersecciones interdependientes.

En su formulación más general, el problema de optimización de control semafórico define como variables de decisión los parámetros temporales que gobiernan la operación del sistema: la duración del ciclo C , los tiempos de verde efectivo asignados a cada fase o movimiento (splits g_i), los desplazamientos temporales entre intersecciones consecutivas (offsets ϕ_j), y en algunos casos la secuencia misma de fases [15, 2]. Estas variables deben elegirse dentro de rangos factibles definidos por la geometría, la capacidad de las aproximaciones y los requerimientos normativos de tiempos mínimos y máximos.

La función objetivo del problema representa la métrica que se desea optimizar y puede adoptar diversas formas según el criterio de desempeño que se privilegie. Entre las formulaciones más comunes se encuentran la minimización del retraso total experimentado por todos los vehículos en la red, la minimización del número total de paradas, la maximización del throughput (número de vehículos que atraviesan el sistema por unidad de tiempo), o la minimización de emisiones contaminantes y consumo de combustible derivados de aceleraciones y frenados [15, 3, 1]. En aplicaciones recientes se han propuesto funciones objetivo multicriterio que balancean simultáneamente eficiencia operativa, equidad entre corrientes de tráfico, seguridad y sostenibilidad ambiental, empleando técnicas de optimización multiobjetivo como frentes de Pareto o funciones de utilidad ponderadas [19, 3].

El problema se completa con un conjunto de restricciones que garantizan la factibilidad y seguridad de las soluciones. Estas restricciones incluyen tiempos mínimos de verde para permitir que los peatones crucen la intersección de forma segura, tiempos máximos de verde para evitar demoras excesivas en movimientos conflictivos, tiempos de amarillo y todo-rojo calculados en función de la velocidad de aproximación y las distancias de despeje, y restricciones de capacidad que limitan los flujos admisibles en cada carril o aproximación [2, 12]. En el caso de redes coordinadas, se añaden restricciones de consistencia de ciclo (todas las intersecciones deben operar con el mismo ciclo común o con múltiplos enteros del ciclo base) y restricciones de sincronización que determinan los valores admisibles de los offsets para lograr progresión de

pelotones [31, 20].

Los métodos analíticos tradicionales, como las fórmulas de Webster [17] o el método del Highway Capacity Manual (HCM) [11], constituyen soluciones cerradas para versiones simplificadas del problema de optimización, en las que se asumen condiciones de estado estacionario, demanda uniforme y geometrías regulares. Estas formulaciones proporcionan estimaciones rápidas del ciclo óptimo y de los splits correspondientes, y resultan útiles para el diseño preliminar de planes de tiempo fijo. Sin embargo, en la práctica, las condiciones reales de tráfico urbano se caracterizan por una alta variabilidad temporal (diferencias entre horas pico y valle, fluctuaciones aleatorias de la demanda), heterogeneidad espacial (diferencias de volumen entre aproximaciones), y la presencia de eventos no recurrentes (accidentes, obras, eventos especiales) que invalidan las hipótesis de estacionariedad de los modelos analíticos [1, 3].

Esta variabilidad convierte al problema de optimización semafórica en un desafío computacional complejo, especialmente cuando se considera la operación coordinada de múltiples intersecciones a lo largo de corredores o redes completas. En estos casos, el espacio de búsqueda de soluciones crece exponencialmente con el número de intersecciones y de fases, y el problema adquiere la estructura de un problema de optimización combinatoria NP-difícil, para el cual no se conocen algoritmos exactos con tiempo de ejecución polinomial [15, 19]. Ante esta complejidad, las técnicas algorítmicas avanzadas, como algoritmos genéticos, Particle Swarm Optimization (PSO), simulated annealing y métodos de búsqueda local, se han consolidado como herramientas eficaces para explorar el espacio de soluciones, escapar de óptimos locales y encontrar configuraciones de alta calidad en tiempos de cómputo razonables [19, 1].

En síntesis, la formulación del control semafórico como problema de optimización combinatoria proporciona un marco conceptual riguroso que permite integrar distintos criterios de desempeño, considerar múltiples restricciones operativas y de seguridad, y justificar el empleo de técnicas computacionales avanzadas, como los sistemas de inferencia difusa y los algoritmos metaheurísticos, que serán presentados en secciones posteriores y que constituyen la base del sistema de control inteligente desarrollado en este trabajo.

2.7. Coordinación de Semáforos y Progresión de Pelotones

En redes urbanas, los semáforos rara vez operan de manera completamente aislada; por el contrario, se busca coordinar varios equipos a lo largo de un corredor para facilitar la progresión de pelotones de vehículos y reducir el número de paradas sucesivas [2, 11]. La coordinación se logra ajustando parámetros como la longitud de ciclo común entre intersecciones, los offsets (desplazamientos temporales entre inicios de verde) y, en algunos casos, la secuencia de fases, de modo que un grupo de vehículos que parte con verde en una intersección encuentre verdes sucesivos en las siguientes, configurando la llamada “onda verde” [12, 31]. Estos esquemas buscan compatibilizar la operación de múltiples nodos de control, sacrificando a veces el óptimo local de una intersección en beneficio del desempeño global del corredor.

La coordinación introduce dependencias fuertes entre los parámetros de distintas intersec-

ciones: cambios en la longitud de ciclo o en los splits de una intersección pueden degradar la progresión en tramos adyacentes, generando nuevas demoras o incrementando el número de paradas en el corredor [31, 11]. Por ello, el diseño de planes coordinados suele apoyarse en herramientas de simulación y optimización que consideran simultáneamente varias intersecciones, y en prácticas de monitoreo operativo que permiten ajustar offsets y ciclos en respuesta a cambios de demanda. En este contexto, la idea de un sistema de control y monitoreo distribuido, apoyado en tecnologías como BlockDAG e IPFS para registrar configuraciones y métricas de desempeño, resulta especialmente pertinente: aunque el presente trabajo se centre en la optimización a nivel de una intersección, la misma arquitectura puede escalar a corredores coordinados donde las decisiones de tiempo de verde, ciclo y offset deben gestionarse de forma conjunta y trazable.

2.8. Consideraciones Operativas y Métricas de Desempeño

La evaluación de un plan de tiempos semafóricos se basa en un conjunto de métricas operativas que permiten cuantificar su impacto sobre los usuarios de la vía. Entre las más utilizadas se encuentran el retraso medio por vehículo, el número de paradas, la longitud de colas y el nivel de servicio (Level of Service, LOS); en estudios recientes se añaden además indicadores de consumo de combustible y emisiones contaminantes [2, 11, 1]. El retraso medio mide la diferencia entre el tiempo de viaje real y el tiempo teórico sin interferencias, mientras que el número de paradas y la longitud de colas caracterizan la discontinuidad del flujo y la probabilidad de bloqueo de accesos o de intersecciones adyacentes [2, 12].

El nivel de servicio resume de forma cualitativa la calidad de operación en categorías que van desde “A” (pocas demoras, operación confortable) hasta “F” (congestión severa), definidas a partir de rangos de retraso medio por vehículo y otras variables [11, 12]. Estas métricas permiten comparar diferentes esquemas de control (planes fijos, control actuado, control adaptativo) bajo condiciones de demanda similares y constituyen la base para justificar cambios en la programación de semáforos desde una perspectiva de ingeniería.

En el contexto de este trabajo, el desempeño de los planes semafóricos se evalúa a partir de indicadores derivados de la salida del sistema difuso de congestión (valor numérico de congestión y su categoría lingüística), junto con variables macroscópicas como volumen por minuto, velocidad media y densidad vehicular. El controlador difuso Mamdani proporciona una medida agregada de congestión a partir de estas variables, y el algoritmo PSO ajusta los tiempos de verde para minimizar directamente dicho índice de congestión y mejorar las condiciones de flujo respecto a una asignación base de tiempos calculada mediante fórmulas clásicas de ingeniería de tráfico. La automatización de la recolección de estas métricas y su análisis en tiempo real plantea desafíos de almacenamiento, trazabilidad e integridad de datos. En sistemas de control inteligente distribuido, donde múltiples nodos (intersecciones) operan de forma coordinada, resulta fundamental contar con mecanismos que aseguren la auditabilidad de las decisiones de control y la inmutabilidad de los registros históricos de configuración y desempeño [32, 33]. Este

requisito motiva el uso de arquitecturas descentralizadas como BlockDAG e IPFS, que serán analizadas en secciones posteriores.

2.9. Integración de Técnicas de Optimización y Control Difuso

La convergencia entre técnicas de optimización metaheurística y sistemas de inferencia difusa ha dado lugar a enfoques híbridos que combinan la capacidad de razonamiento aproximado y manejo de incertidumbre de la lógica difusa con la potencia de búsqueda y ajuste paramétrico de algoritmos como Particle Swarm Optimization (PSO), algoritmos genéticos y otras metaheurísticas [1, 34]. En el contexto del control semafórico, estos sistemas híbridos fuzzy–metaheurísticos permiten abordar de manera integrada dos desafíos complementarios: por un lado, la captura y formalización del conocimiento experto sobre las relaciones entre variables de tráfico y decisiones de control mediante reglas lingüísticas y funciones de pertenencia difusas; por otro, la optimización automática de los parámetros del sistema difuso (formas y umbrales de las funciones de pertenencia, pesos de las reglas, ganancias de salida) de forma que se maximice una función objetivo operativa, como la minimización de retrasos o la maximización del throughput de la red [35, 36].

En un sistema de control difuso tipo Mamdani aplicado a semáforos, las funciones de pertenencia de las variables de entrada (por ejemplo, volumen vehicular, velocidad media, densidad, longitud de cola) y de salida (extensión de verde, ajuste de ciclo, cambio de fase) se definen inicialmente con base en conocimiento experto o en datos históricos, pero su sintonización fina puede requerir ajustes iterativos costosos y poco sistemáticos. PSO ofrece una alternativa eficiente y automática: cada partícula del enjambre representa una configuración específica de parámetros del sistema difuso, y la posición de la partícula se actualiza iterativamente en función de su desempeño histórico y del desempeño del mejor individuo global, explorando el espacio de configuraciones hasta encontrar un conjunto de parámetros que optimice la métrica de desempeño deseada [7, 23, 24]. Este enfoque de optimización de parámetros difusos mediante PSO ha demostrado reducir significativamente el tiempo de diseño del controlador, mejorar la robustez frente a variaciones de demanda y permitir la adaptación online de parámetros en respuesta a condiciones cambiantes [34, 35].

Extensiones más complejas incluyen sistemas neuro–fuzzy, en los que se combinan redes neuronales artificiales con lógica difusa para permitir aprendizaje automático de las reglas de inferencia y funciones de pertenencia a partir de datos, y que pueden ser entrenados o ajustados mediante metaheurísticas como PSO, algoritmos genéticos o búsqueda cuco (Cuckoo Search) [37]. Estos enfoques híbridos neuro–fuzzy–metaheurísticos aprovechan la capacidad de generalización y aprendizaje de las redes neuronales, la interpretabilidad de los sistemas difusos y la eficiencia de búsqueda de las metaheurísticas, constituyendo una vía prometedora para el diseño de controladores adaptativos en dominios complejos como el control de tráfico urbano

[1, 36].

El sistema propuesto en este trabajo se inscribe en esta línea de sistemas híbridos fuzzy-PSO: emplea un controlador difuso tipo Mamdani para evaluar el nivel de congestión a partir de variables macroscópicas de tráfico (volumen, velocidad, densidad) y utiliza PSO para optimizar los tiempos de verde de las fases semafóricas de forma que se minimice el índice de congestión estimado por el sistema difuso. Este enfoque permite incorporar conocimiento experto sobre la relación entre variables de tráfico y estados de congestión en las reglas difusas, mientras que PSO asegura que la asignación de tiempos de verde sea óptima respecto de la configuración de demanda observada. La arquitectura resultante es adaptativa, interpretable y computacionalmente eficiente, características esenciales para su despliegue en entornos operativos reales.

Más allá de las ventajas operativas del enfoque híbrido, la implementación de sistemas de control inteligente en infraestructuras críticas como redes semafóricas plantea requisitos adicionales de trazabilidad, auditabilidad e integridad de datos. En un sistema distribuido donde múltiples intersecciones operan de forma coordinada o autónoma, resulta fundamental registrar de manera inmutable y verificable las decisiones de control tomadas (tiempos de verde asignados, parámetros del controlador difuso, resultados de la optimización PSO), las condiciones de tráfico observadas (volúmenes, velocidades, densidades capturadas por sensores) y las métricas de desempeño resultantes (retrasos, emisiones, niveles de servicio). Estos registros permiten auditar el comportamiento del sistema, identificar configuraciones problemáticas, validar la conformidad con normativas y políticas de operación, y facilitar la depuración y mejora continua del controlador. Las arquitecturas descentralizadas basadas en BlockDAG (Directed Acyclic Graph de bloques) e IPFS (InterPlanetary File System) ofrecen soluciones escalables y confiables para el registro y almacenamiento distribuido de estos datos, temas que serán analizados en profundidad en secciones posteriores del presente trabajo. En síntesis, la integración de técnicas de optimización metaheurística y control difuso no solo mejora el desempeño operativo del sistema semafórico, sino que abre la puerta a arquitecturas distribuidas y auditables que responden a las demandas de transparencia y confiabilidad de las ciudades inteligentes modernas.

CAPÍTULO 3

VISIÓN GENERAL DEL SISTEMA

El sistema propuesto se concibe como un ecosistema modular de microservicios para el monitoreo, análisis y optimización del tráfico urbano en tiempo (casi) real, diseñado para operar en entornos de ciudad inteligente. La arquitectura se compone de cuatro módulos principales, cada uno con responsabilidades bien definidas y comunicándose mediante interfaces estandarizadas (APIs REST y mensajes JSON). Esta organización modular facilita la escalabilidad, el mantenimiento independiente de componentes y la integración con fuentes de datos heterogéneas.

3.1. Módulos Principales

El sistema se estructura en torno a cuatro módulos fundamentales:

- **traffic-sim**: Módulo de simulación de tráfico urbano, que genera escenarios de prueba controlados y reproducibles para evaluar el comportamiento del sistema bajo diferentes condiciones de demanda vehicular. Permite emular redes viales con semáforos y flujos vehiculares variables.
- **traffic-control**: Módulo de orquestación que actúa como punto de entrada unificado al sistema. Recibe observaciones de tráfico, valida los datos, coordina las invocaciones a los servicios especializados de optimización y almacenamiento, y mantiene metadatos operativos en una base de datos relacional.
- **traffic-sync**: Módulo de análisis y optimización que procesa las métricas de tráfico recibidas, evalúa el nivel de congestión mediante técnicas de inteligencia computacional, y genera configuraciones optimizadas de tiempos semafóricos que buscan reducir demoras y mejorar el flujo vehicular.

- **traffic-storage:** Módulo de persistencia verificable que almacena datos del sistema en infraestructuras descentralizadas, garantizando trazabilidad, integridad e inmutabilidad de los registros históricos mediante el uso de tecnologías de almacenamiento distribuido y registro de auditoría.

3.2. Flujo General de Información

El flujo operativo del sistema sigue un ciclo cerrado de captura, análisis, optimización y registro:

1. **Captura de datos:** El módulo de simulación (o sensores reales en un despliegue futuro) genera observaciones de tráfico que incluyen métricas agregadas por intersección semafórizada.
2. **Validación y orquestación:** El módulo de control recibe las observaciones, valida su estructura y coherencia, y coordina el flujo de información hacia los servicios especializados.
3. **Análisis y optimización:** El módulo de sincronización procesa las métricas, clasifica el estado de congestión y determina configuraciones de tiempos semafóricos que mejoren el desempeño operativo.
4. **Persistencia verificable:** Los datos originales y los resultados de optimización se almacenan de forma distribuida, registrando identificadores de contenido en estructuras inmutables que permiten auditar y verificar el comportamiento histórico del sistema.

3.3. Justificación de la Arquitectura

La elección de una arquitectura de microservicios sobre un diseño monolítico responde a requisitos concretos del sistema. Los cuatro módulos tienen ciclos de vida y stacks tecnológicos radicalmente distintos: traffic-sim depende de SUMO y librerías de red vial; traffic-sync combina scikit-fuzzy con optimización metaheurística; traffic-storage requiere un cliente web3 y el daemon de IPFS; traffic-control actúa exclusivamente como orquestador sin procesamiento intensivo. Empaquetar estos cuatro dominios en un solo proceso eliminaría la posibilidad de reemplazar, por ejemplo, el simulador por sensores IoT reales sin recompilar el núcleo analítico.

La comunicación mediante REST/JSON se eligió frente a alternativas binarias como gRPC porque las latencias dominantes del sistema son la carga a IPFS ($\approx 1,2$ s) y la confirmación en BlockDAG ($\approx 3,5$ s), por lo que la sobrecarga de serialización JSON no es el cuello de botella. REST también reduce la barrera de integración con infraestructura municipal existente y con sensores IoT estándar que ya exponen APIs HTTP.

La división en exactamente cuatro módulos refleja cuatro responsabilidades sin superposición: generar datos (traffic-sim), coordinar y validar (traffic-control), analizar y optimizar

(traffic-sync), y persistir con verificabilidad (traffic-storage). Fusionar, por ejemplo, orquestación y optimización acoplaría la lógica de validación de entradas con el ciclo difuso-PSO, dificultando las pruebas independientes de cada componente.

Las propiedades que resultan de estas decisiones son:

- **Bajo acoplamiento:** Los módulos se comunican exclusivamente mediante contratos REST/JSON; evolucionar o reemplazar un componente no requiere modificar los demás.
- **Escalabilidad:** traffic-sync escala linealmente con el número de sensores ($\approx 0,034$ s/sensor hasta 2000 sensores), como se demuestra en el Capítulo 8.
- **Trazabilidad verificable:** cada lote de datos recibe un CID de IPFS inmutable, registrado en el contrato `TrafficStorage.sol`, lo que permite auditar retroactivamente cualquier decisión de control.
- **Reproducibilidad:** SUMO genera escenarios con semillas deterministas, garantizando que los experimentos del Capítulo 8 puedan reproducirse de forma independiente.
- **Interoperabilidad:** la interfaz REST de traffic-control permite sustituir traffic-sim por sensores físicos sin cambios en los módulos de análisis ni de almacenamiento.

La Figura 3.1 ilustra de forma esquemática la organización modular del sistema, mostrando los cuatro módulos principales y sus relaciones.

En síntesis, esta arquitectura modular proporciona una plataforma flexible y escalable para la gestión inteligente de tráfico urbano, combinando capacidades de simulación, análisis adaptativo y persistencia verificable en un sistema distribuido que puede integrarse con infraestructuras reales de monitoreo vehicular. Los capítulos subsiguientes describen en detalle la implementación, las tecnologías subyacentes y el funcionamiento de cada módulo.

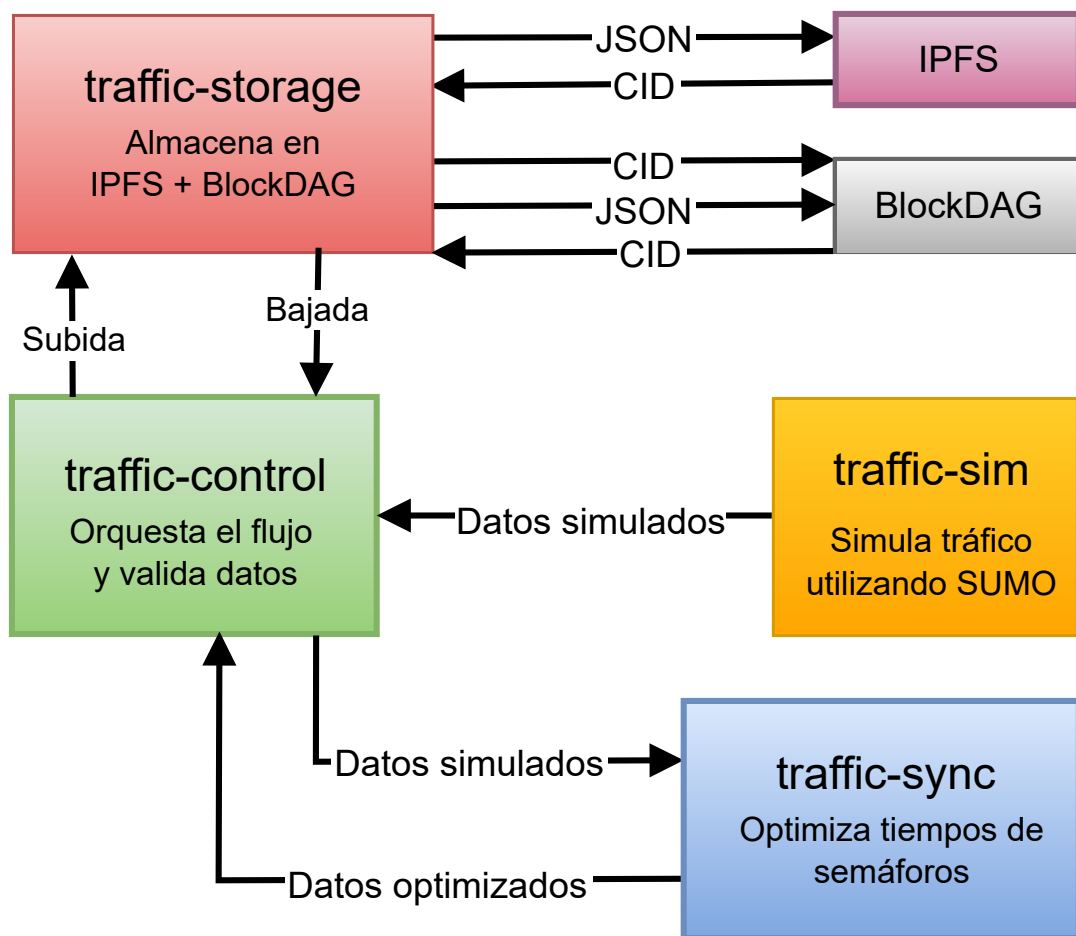


Figura 3.1: Arquitectura general del sistema mostrando los cuatro módulos principales y el flujo de información.

CAPÍTULO 4

SIMULACIÓN DE TRÁFICO

El módulo `traffic-sim` implementa un entorno de simulación de tráfico urbano basado en SUMO (Simulation of Urban MObility), utilizado para evaluar el comportamiento del sistema propuesto bajo escenarios controlados y reproducibles. SUMO es un simulador microscópico de tráfico ampliamente adoptado en la comunidad científica, que permite modelar redes viales, rutas de vehículos y programas semafóricos, y ejecutar simulaciones paso a paso mediante una interfaz de control remoto (`traci`) [38]. En este trabajo, `traffic-sim` se integra con el servicio de optimización `traffic-sync` para cerrar el ciclo “simulación–detección–optimización–aplicación” sobre redes generadas a partir de datos reales o escenarios de prueba.

4.1. Arquitectura del Módulo

La estructura de `traffic-sim` se organiza en torno a un orquestador de simulación y un conjunto de componentes especializados para detección, control y comunicación externa. El orquestador de simulación encapsula la lógica principal: carga la configuración de SUMO (archivos de configuración de red, aristas, nodos, rutas y semáforos), inicializa la simulación mediante la interfaz de control remoto y gestiona el avance de la simulación en pasos discretos [38]. El punto de entrada recibe como argumento un archivo comprimido de escenario (generado, por ejemplo, mediante la herramienta auxiliar de construcción de mapas) y ejecuta la simulación en modo sin interfaz gráfica o con interfaz gráfica.

Los detectores de cuellos de botella monitorean métricas como densidad de vehículos, velocidad promedio y longitud de cola por tramo o aproximación. Los umbrales y parámetros de detección (por ejemplo, densidad mínima, velocidad máxima para considerar congestión, intervalo entre detecciones, duración mínima del evento) se centralizan en la configuración del sistema, lo que permite ajustar la sensibilidad del sistema sin modificar la lógica de detección.

El controlador de semáforos expone métodos para actualizar dinámicamente los tiempos de verde y de ciclo de los semáforos modelados en SUMO, aplicando las optimizaciones recibidas desde el servicio de optimización.

El cliente HTTP se comunica con la API del módulo de optimización: cuando se detecta un cuello de botella, este cliente construye una carga útil con métricas agregadas de tráfico para la intersección crítica y lo envía al punto de entrada correspondiente (por ejemplo, `/evaluate`). Las utilidades de métricas agregan y validan las variables macroscópicas de SUMO (viajes completados, tiempos de viaje, velocidades y colas), garantizando que la información enviada al módulo de optimización siga el esquema esperado por el sistema difuso y el optimizador PSO.

Las herramientas de visualización proporcionan análisis posterior de resultados: parsers de archivos de salida de SUMO (información de viajes, resúmenes, datos de vehículos), funciones para generar gráficos de distribución de tiempos de viaje, evolución temporal de congestión y comparaciones entre diferentes configuraciones de semáforos, así como envoltorios sobre herramientas nativas de SUMO. Estas utilidades permiten cuantificar el impacto de las estrategias de control sobre métricas clave como tiempo de viaje, tiempos de espera y niveles de congestión.

La Figura 4.1 ilustra la arquitectura del módulo traffic-sim, mostrando los componentes principales y sus interacciones con SUMO y el servicio traffic-sync.

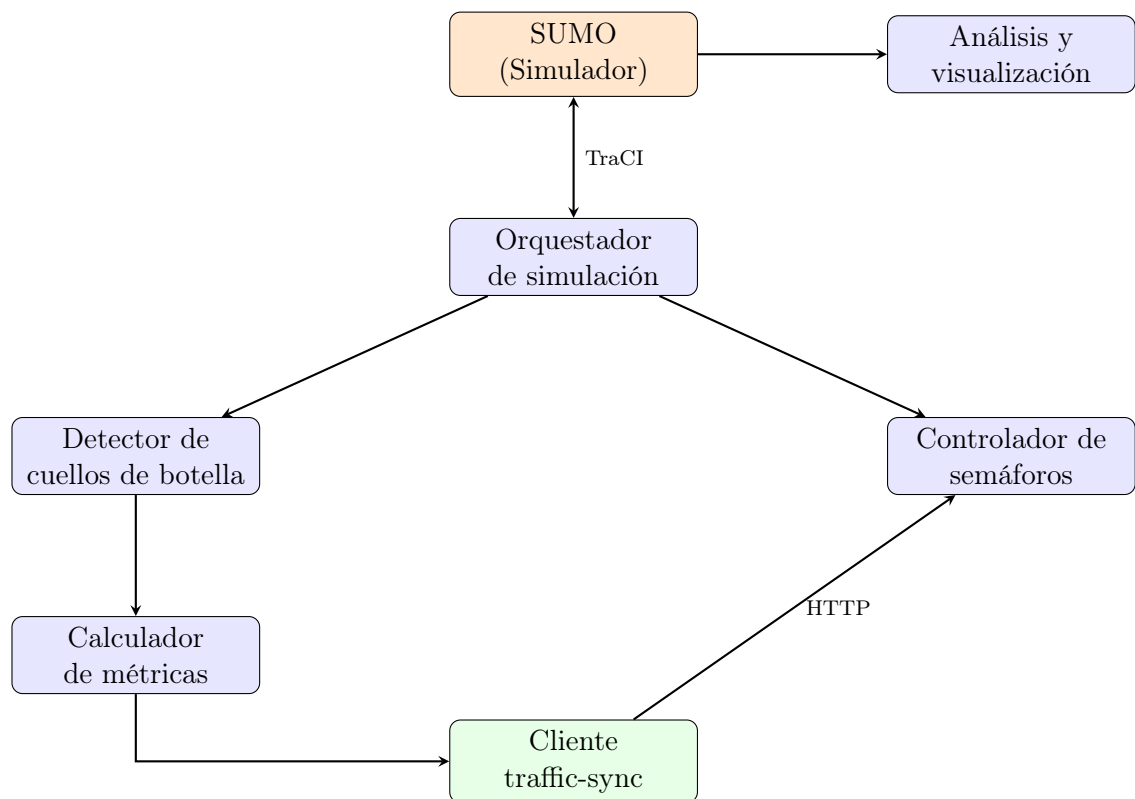


Figura 4.1: Arquitectura del módulo traffic-sim y sus componentes principales [38].

4.2. Flujo de Integración

El flujo de integración entre traffic-sim y traffic-sync sigue una secuencia cíclica que replica el comportamiento de un sistema de control de tráfico en línea:

1. **Ejecución de la simulación:** el orquestador inicia SUMO con el escenario suministrado y avanza la simulación paso a paso, según las rutas y patrones de demanda definidos en los archivos de configuración.
2. **Detección de cuellos de botella:** en intervalos configurables, el detector analiza para cada aproximación variables como densidad, velocidad y longitud de cola, y decide si existe un cuello de botella según los umbrales establecidos en la configuración del sistema.
3. **Construcción de métricas de tráfico:** cuando se confirma un cuello de botella en una intersección con semáforo, se calcula un conjunto de métricas agregadas para esa localización (vehículos por minuto, velocidad media en km/h, tiempo medio de circulación, densidad y estadísticos por tipo de vehículo), que se empaquetan en una carga útil JSON siguiendo el formato consumido por el módulo de optimización.
4. **Comunicación con el módulo de optimización:** el cliente HTTP envía la carga útil al servicio de optimización mediante una petición HTTP (por ejemplo, al punto de entrada /`evaluate`), y espera la respuesta con los tiempos de verde optimizados y el impacto estimado sobre la congestión.
5. **Aplicación de optimizaciones:** el controlador de semáforos actualiza, a través de la interfaz de control remoto, los tiempos de verde y rojo de los programas semafóricos correspondientes en la simulación SUMO, utilizando los valores recibidos desde el módulo de optimización.
6. **Continuación de la simulación:** la simulación se reanuda con los nuevos parámetros de control y se repite el proceso de detección y optimización mientras existan vehículos en red o hasta alcanzar el tiempo límite definido.

De esta manera, traffic-sim se comporta como un laboratorio virtual donde se evalúa en bucle el desempeño del controlador difuso Mamdani y del optimizador PSO, con escenarios contruidos a partir de mapas reales. Los resultados, incluyendo métricas temporales y distribuciones de tiempos de viaje, se analizan mediante las utilidades de visualización.

4.3. Escenario de Simulación y Parámetros de Detección

Red vial. La red se construyó a partir de datos de OpenStreetMap para un sector del área metropolitana de Asunción, Paraguay (latitud $-25,34$ – $-25,35$ S, longitud $-57,50$ – $-57,51$ O),

abarcando aproximadamente 0,8 km². La red comprende 22 nodos, 33 tramos y 7 intersecciones semaforizadas, con vehículos clasificados en automóvil, motocicleta, ómnibus y camión.

Demanda vehicular. Se inyectaron 5 000 vehículos con tiempos de salida distribuidos uniformemente en el horizonte de simulación (0–3 600 s), correspondiendo a una tasa media de ≈ 83 vehículos/minuto. La simulación avanza en pasos de 1 s durante 3 600 pasos (1 hora de tráfico urbano), con un máximo de 1 000 vehículos simultáneos en red.

Configuración semafórica base. Cada semáforo opera inicialmente con un ciclo de 90 s, tiempos de verde ajustables entre 10 s y 120 s por fase, 3 s de luz amarilla y 1 s de todo-rojo entre fases conflictivas.

Umbral del detector de cuellos de botella. El detector analiza cada intersección cada 15 pasos de simulación (15 s) y confirma un evento solo si la condición persiste al menos 3 s. Se requiere un mínimo de 4 vehículos presentes y al menos dos de las siguientes condiciones:

- Densidad > 50 veh/km.
- Velocidad media < 15 km/h.
- Cola ≥ 3 vehículos detenidos (< 1 m/s).

La severidad se clasifica como *alta* (densidad elevada y velocidad baja), *media* (densidad elevada y cola significativa, o velocidad baja y cola significativa) o *baja* (solo densidad elevada o velocidad muy reducida).

Carga útil hacia el módulo de optimización. Al detectar un cuello de botella, el cliente HTTP construye una carga útil JSON con el esquema de la Tabla 4.1.

Cuadro 4.1: Esquema del payload enviado de traffic-sim a traffic-sync ante un cuello de botella.

Campo	Tipo	Descripción / Ejemplo
version	string	Versión del protocolo ("2.0")
type	string	Tipo de mensaje ("data")
timestamp	string	ISO 8601 UTC ("2025-09-12T14:23:01")
traffic_light_id	string	ID del semáforo SUMO ("20928101")
sensors[].controlled_edges	array	Tramos controlados ([143977090#0", ...])
sensors[].metrics.vehicles_per_minute	int	Volumen observado (18)
sensors[].metrics.avg_speed_kmh	float	Velocidad media en km/h (12.4)
sensors[].metrics.avg_circulation_time_sec	float	Tiempo medio de circulación en s (47.3)
sensors[].metrics.density	float	Densidad en veh/km (67.2)
sensors[].vehicle_stats	object	Conteo por tipo: {car, motorcycle, b...

CAPÍTULO 5

ALMACENAMIENTO DISTRIBUIDO Y TECNOLOGÍAS DESCENTRALIZADAS

El módulo traffic-storage actúa como capa de persistencia verificable del sistema, recibiendo métricas de tráfico y resultados de optimización, almacenando los datos en un repositorio distribuido y registrando huellas inmutables en infraestructuras descentralizadas. En este contexto, blockchain, BlockDAG e IPFS se utilizan de forma complementaria: blockchain y BlockDAG como ledgers inmutables para trazabilidad, e IPFS como almacén de archivos orientado a contenido [39, 8, 9].

5.1. Blockchain

Blockchain puede describirse como un libro mayor distribuido donde las transacciones se agrupan en bloques enlazados criptográficamente en una cadena lineal, de modo que cada bloque referencia al anterior mediante un resumen criptográfico (hash), proporcionando inmutabilidad y resistencia a manipulaciones retrospectivas [39, 40]. Los nodos de la red ejecutan un mecanismo de consenso (como Proof-of-Work o Proof-of-Stake) para acordar qué bloques se consideran válidos, garantizando que todos mantengan una vista consistente del historial de transacciones [41, 42].

La Figura 5.1 ilustra la estructura básica de una cadena de bloques, mostrando cómo cada bloque referencia criptográficamente al bloque anterior mediante su resumen criptográfico (hash), creando una cadena inmutable donde cualquier modificación en un bloque anterior invalida todos los bloques subsiguientes.

En este trabajo, blockchain se utiliza para registrar metadatos de almacenamiento de tráfico de forma inmutable, actuando como capa de “prueba de existencia” para datos que residen físicamente fuera de la cadena. El contrato inteligente de almacenamiento expone operaciones

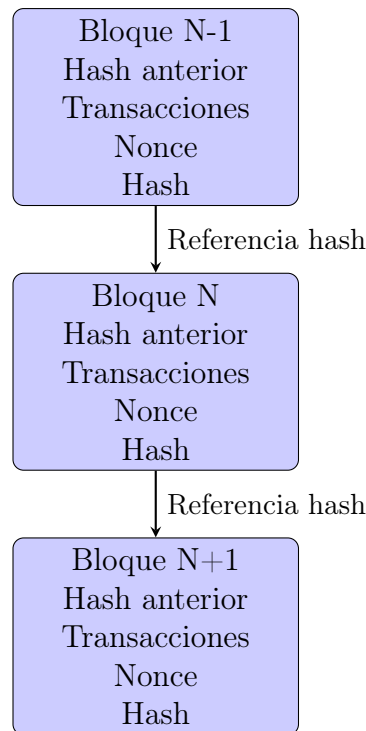


Figura 5.1: Estructura básica de una cadena de bloques, mostrando la dependencia criptográfica entre bloques consecutivos [39].

de alto nivel para registrar eventos de almacenamiento: cada llamada almacena, al menos, un identificador de semáforo, una marca de tiempo y un identificador de contenido (CID de IPFS), junto con el tipo de dato al que hace referencia. Este contrato se despliega y prueba utilizando un entorno de desarrollo estándar, con scripts de despliegue y pruebas automatizadas que verifican su funcionamiento básico.

El módulo `traffic-storage` ofrece una API HTTP que recibe cargas de datos en un formato unificado (versión 2.0), generadas por otros componentes del sistema (como los módulos de optimización y lógica difusa). Cuando llega una nueva medición o reporte, la API valida la carga útil, delega la persistencia de contenido a IPFS y, una vez obtenido el CID, invoca al contrato inteligente de almacenamiento para registrar ese CID junto con los metadatos relevantes en la blockchain. De este modo, los algoritmos de optimización y los módulos de monitoreo solo interactúan con una interfaz REST, mientras que los detalles de interacción on-chain quedan encapsulados en el módulo de almacenamiento.

5.2. BlockDAG

BlockDAG (Block Directed Acyclic Graph) generaliza la estructura lineal de blockchain permitiendo que cada bloque referencie a múltiples bloques padres anteriores, formando un grafo dirigido acíclico en lugar de una cadena única [43, 9]. Esta arquitectura permite que bloques creados en paralelo se integren en el consenso sin ser descartados como huérfanos, mejorando el aprovechamiento del ancho de banda de la red y posibilitando mayores tasas de

transacciones y menores latencias de confirmación que las típicamente alcanzables en cadenas lineales [43].

La Tabla 5.1 sintetiza las principales diferencias entre arquitecturas blockchain tradicionales y BlockDAG basadas en protocolos como GHOSTDAG. La comparación considera protocolos representativos de cada paradigma: Nakamoto consensus implementado en Bitcoin y Ethereum para blockchains lineales [39, 41], y PHANTOM/GHOSTDAG como generalización del consenso distribuido en estructuras DAG [9].

Cuadro 5.1: Comparación entre Blockchain Tradicional y BlockDAG

Característica	Blockchain Tradicional	BlockDAG (GHOSTDAG)
Topología	Lineal (1 padre)[9]	Grafo acíclico (múltiples padres)[9, 44]
Bloques huérfanos	Descartados[9]	Integrados como padres[9]
Ordenamiento	Total (intrínseco)	Parcial $\rightarrow$ Total (vía algoritmo)[9]
Escalabilidad	Limitada[9]	Alta (ancho de banda físico)[9, 44]
Confirmación	Probabilística lenta	Probabilística rápida (segundos)[44]
Minería	Competitiva[9]	Cooperativa[9]
Throughput típico	7-30 TPS[39, 41]	100+ TPS[44]
Latencia de bloque	10 min (Bitcoin), 12 s (Ethereum)[45]	Sub-segundo a segundos[44]

En el contexto de este sistema, BlockDAG se considera como infraestructura de registro distribuido complementaria a la blockchain tradicional, pensada para escenarios de mayor escala donde múltiples intersecciones reportan datos con alta frecuencia. El módulo traffic-storage incorpora una capa de abstracción que permite, según la configuración, registrar eventos de almacenamiento tanto en un contrato EVM como en una red BlockDAG compatible, utilizando un cliente dedicado que construye y envía mensajes de registro a un nodo remoto. Estos mensajes contienen, de forma análoga a la variante blockchain, el identificador de semáforo, la marca de tiempo y el resumen criptográfico de contenido (CID de IPFS u otro identificador criptográfico).

La lógica de alto nivel en el módulo de almacenamiento se mantiene independiente de la tecnología subyacente: la API recibe métricas y resultados de optimización, delega la persistencia de archivos a IPFS y, posteriormente, invoca una interfaz de registro de evidencia"que puede mapearse a blockchain, BlockDAG o a un esquema dual. Esta separación permite que, a medida que se exploren o adopten redes DAG de mayor throughput para despliegues reales en entornos de ciudad inteligente, el resto del sistema (clasificación de congestión, PSO, paneles de monitoreo) continúe interactuando con un mismo punto de entrada consistente.

5.3. IPFS

El InterPlanetary File System (IPFS) es un sistema de archivos distribuido peer-to-peer que utiliza direccionamiento por contenido: cada objeto se identifica por un Content Identifier (CID) que codifica un resumen criptográfico (hash) del contenido más metadatos mínimos sobre cómo interpretarlo [8]. El mismo archivo siempre produce el mismo CID, independientemente

del nodo que lo procese, lo que permite verificar integridad de manera determinista y realizar deduplicación automática a escala de red [46].

La Figura 5.2 ilustra el flujo básico de almacenamiento y recuperación en IPFS, mostrando cómo un archivo se fragmenta, se le asigna un CID basado en su contenido, y puede ser recuperado desde cualquier nodo que posea los bloques correspondientes.

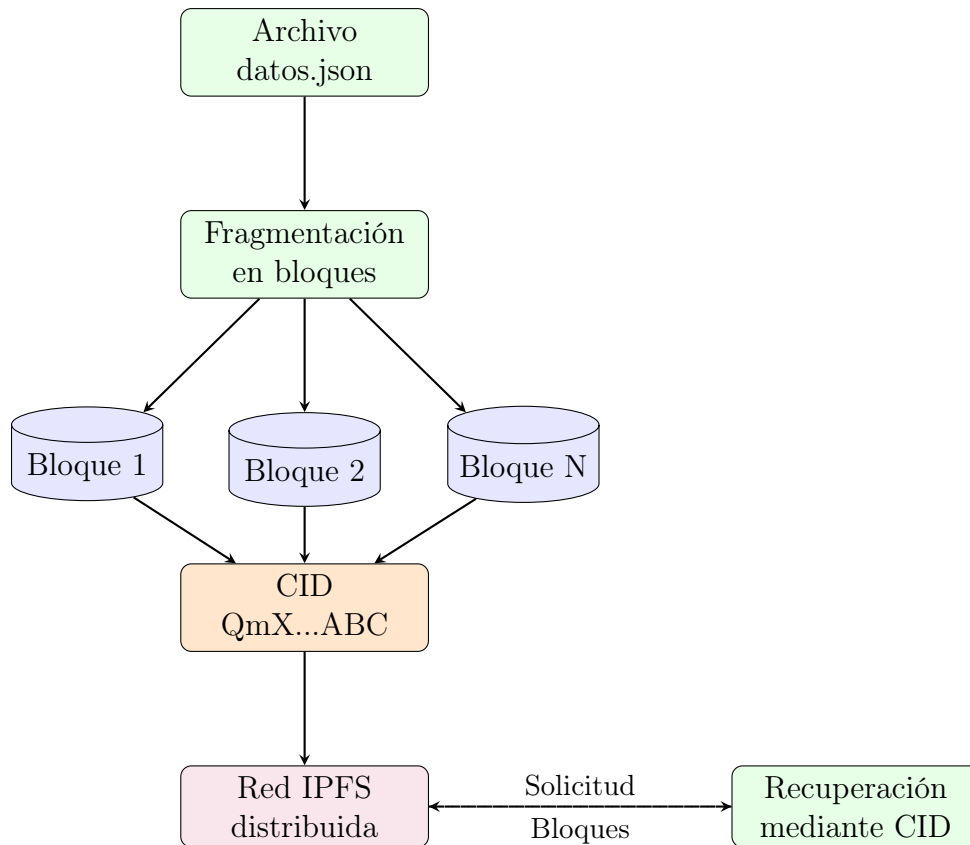


Figura 5.2: Arquitectura básica de IPFS: fragmentación de archivos, generación de CID y recuperación distribuida [8].

En este trabajo, IPFS se emplea como almacén de datos off-chain para los artefactos generados y consumidos por el sistema: mediciones agregadas de tráfico por grupo (clúster), indicadores de congestión difusa, configuraciones optimizadas de tiempos semafóricos y reportes experimentales. El módulo traffic-storage incluye un cliente específico para interactuar con un nodo IPFS local o con un servicio compatible, ofreciendo operaciones sencillas para subir y recuperar objetos. El flujo básico consiste en que la API recibe una carga útil de datos, lo serializa (por ejemplo, como JSON), lo envía al nodo IPFS y obtiene de vuelta un CID.

Ese CID se utiliza a continuación como enlace entre IPFS y las capas de consenso: se registra en blockchain o en BlockDAG mediante las interfaces descritas anteriormente, de modo que cualquier módulo que necesite auditar o reanalizar un conjunto de datos solo requiera conocer el CID y el identificador de semáforo asociado. Desde la perspectiva del resto del sistema, IPFS y las tecnologías de registro inmutable quedan encapsuladas detrás del módulo de almacenamiento: los módulos de simulación, clasificación difusa y optimización interactúan únicamente con la API REST, mientras que las garantías de verificabilidad, integridad y persistencia se logran

combinando direccionamiento por contenido en IPFS con registros inmutables en blockchain y/o BlockDAG.

La Figura 5.3 presenta un diagrama de secuencia que ilustra el flujo completo de almacenamiento distribuido: desde la recepción de datos por la API hasta el registro del CID en la capa de consenso, mostrando la interacción entre los componentes del módulo de almacenamiento.

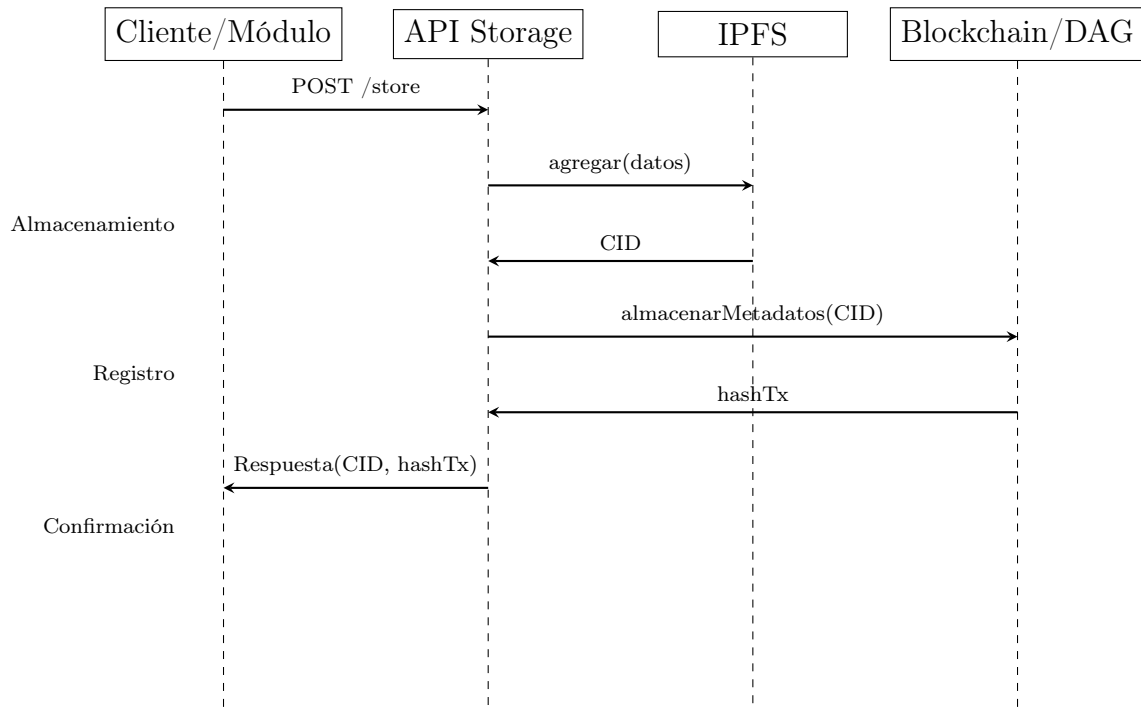


Figura 5.3: Diagrama de secuencia del flujo de almacenamiento distribuido en traffic-storage.

5.4. Interfaz del Contrato TrafficStorage

El contrato inteligente `TrafficStorage` (Solidity ^0.8.19) es la única pieza de código desplegada on-chain en el sistema. Su diseño deliberadamente minimalista reduce la huella de gas al mínimo: no almacena datos en bruto sino únicamente los CID de IPFS junto con tres claves de búsqueda.

Tipos. El enum `DataType` distingue dos categorías de registros:

- `Data` (valor 0): lote de métricas de tráfico sin procesar.
- `Optimization` (valor 1): resultado del ciclo difuso-PSO (tiempos de verde óptimos).

El almacenamiento interno es un mapeo triple privado:

```
mapping(string =>mapping(uint256 =>mapping(DataType =>string))) private records;
```

indexado por `(trafficLightId, timestamp, dataType)`, lo que permite acceso $O(1)$ a cualquier registro conociendo las tres claves.

Funciones y evento. La Tabla 5.2 describe la interfaz pública completa del contrato.

Cuadro 5.2: Interfaz pública del contrato `TrafficStorage.sol`.

Nombre	Tipo	Descripción
<code>storeRecord(</code> <code>string</code> <code>trafficLightId,</code> <code>uint256</code> <code>timestamp,</code> <code>DataType</code> <code>dataType,</code> <code>string cid)</code>	<code>external</code>	Escribe <code>cid</code> bajo la clave compuesta (<code>trafficLightId</code>, <code>timestamp</code>, <code>dataType</code>) y emite <code>RecordStored</code>. Sin modificador de acceso: cualquier cuenta puede invocarla.
<code>getRecord(</code> <code>string</code> <code>trafficLightId,</code> <code>uint256</code> <code>timestamp,</code> <code>DataType</code> <code>dataType)</code>	<code>external view</code>	Retorna el CID almacenado para la clave compuesta. Revierte con "Record not found" si no existe entrada para esa combinación.
<code>RecordStored(</code> <code>string</code> <code>trafficLightId,</code> <code>uint256</code> <code>timestamp,</code> <code>DataType</code> <code>dataType,</code> <code>string cid)</code>	<code>event</code>	Emitido por <code>storeRecord</code> al completarse. Los indexadores de la red pueden filtrar eventos por semáforo o por tipo para reconstruir el historial de una intersección.

Estimaciones de gas. Medidas sobre la red BlockDAG de prueba con las cargas reales del sistema (véase Capítulo 8), la llamada a `storeRecord` consume en promedio 36 232 unidades de gas para registros de tipo `Data` y 95 624 unidades para registros de tipo `Optimization`. La diferencia refleja el mayor tamaño del CID de IPFS en los paquetes de optimización, que contienen vectores de tiempos de verde además de los metadatos. La función `getRecord` es de solo lectura (`view`) y no incurre en costo de gas cuando se invoca off-chain.

CAPÍTULO 6

OPTIMIZACIÓN Y SINCRONIZACIÓN DE TRÁFICO

El módulo `traffic-sync` implementa el servicio de optimización de tráfico del sistema, exponiendo una API que recibe mediciones macroscópicas de tráfico por semáforo y devuelve configuraciones de tiempos optimizadas. Para ello combina dos componentes principales: un sistema de inferencia difusa tipo Mamdani, que evalúa el nivel de congestión a partir de variables de flujo, velocidad y densidad; y un algoritmo de optimización Particle Swarm Optimization (PSO), que ajusta dinámicamente los tiempos de verde con el objetivo de reducir la congestión estimada [25, 6, 7].

6.1. Lógica Difusa Mamdani

La lógica difusa proporciona un marco formal para representar razonamiento humano impreciso mediante variables lingüísticas y conjuntos difusos con grados de pertenencia entre 0 y 1 [6, 47]. En un sistema de inferencia Mamdani, las reglas tienen la forma “SI condición ENTONCES acción”, donde tanto antecedentes como consecuentes se modelan como conjuntos difusos, y la salida final se obtiene tras un proceso de agregación y defuzzificación [25, 27]. Este enfoque resulta especialmente adecuado para describir estados de tráfico como “flujo libre”, “denso” o “congestionado” a partir de variables macroscópicas continuas [28, 26].

La Figura 6.1 ilustra un ejemplo de variable lingüística para velocidad promedio con tres términos (baja, media, alta) definidos mediante funciones de membresía, mostrando cómo un mismo valor numérico puede tener diferentes grados de pertenencia a múltiples categorías lingüísticas.

En `traffic-sync`, el sistema de inferencia difusa se implementa en el módulo de lógica difusa, que está compuesto principalmente por el módulo de definición del sistema y el módulo de

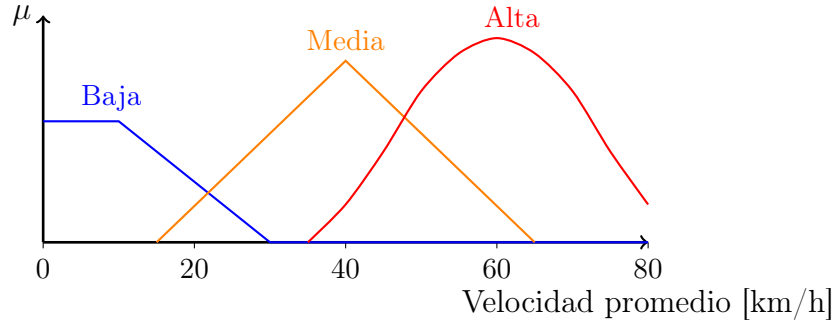


Figura 6.1: Variable lingüística *velocidad promedio* con tres conjuntos difusos modelados mediante funciones de membresía trapezoidal, triangular y aproximadamente gaussiana [27].

evaluación. El módulo de definición del sistema define las variables lingüísticas de entrada (por ejemplo, vehículos por minuto, velocidad media, tiempo medio de circulación, densidad) y la variable de salida asociada al nivel de congestión, junto con sus funciones de membresía triangulares y trapezoidales calibradas en base a datos de tráfico y criterios de ingeniería [27, 28]. La base de reglas Mamdani captura relaciones cualitativas del tipo “SI flujo es alto Y velocidad es baja ENTONCES congestión es severa”, que reflejan tanto conocimiento experto como patrones observados en los datos [26].

La Tabla 6.1 especifica los parámetros exactos de cada función de membresía implementada.

Cuadro 6.1: Parámetros de las funciones de membresía del sistema difuso Mamdani en traffic-sync

Variable	Término	Tipo	Parámetros
Vehículos/min [0–50]	Bajo	Trapezoidal	[0, 0, 15, 25]
	Medio	Gaussiana	$\mu = 30, \sigma = 5$
	Alto	Trapezoidal	[35, 40, 50, 50]
Velocidad km/h [0–60]	Baja	Trapezoidal	[0, 0, 20, 30]
	Media	Gaussiana	$\mu = 35, \sigma = 5$
	Alta	Trapezoidal	[40, 50, 60, 60]
Densidad veh/km [0–150]	Baja	Trapezoidal	[0, 0, 50, 80]
	Media	Gaussiana	$\mu = 100, \sigma = 15$
	Alta	Trapezoidal	[120, 130, 150, 150]
Congestión [0–10]	Ninguna	Trapezoidal	[0, 0, 2, 4]
	Leve	Triangular	[3, 5, 7]
	Severa	Trapezoidal	[6, 8, 10, 10]

La Figura 6.2 resume el flujo de procesamiento en un sistema de inferencia Mamdani aplicado al problema de congestión vehicular, mostrando las etapas desde la fuzzificación de entradas hasta la defuzzificación que produce el nivel de congestión final.

El módulo de evaluación expone funciones de alto nivel que reciben vectores de métricas de tráfico en formato estructurado y devuelven, para cada sensor o semáforo, un valor numérico de congestión y su categoría lingüística asociada. Estas funciones se utilizan en el pipeline principal del servicio web: el punto de entrada `/evaluate` recibe un lote de sensores, construye

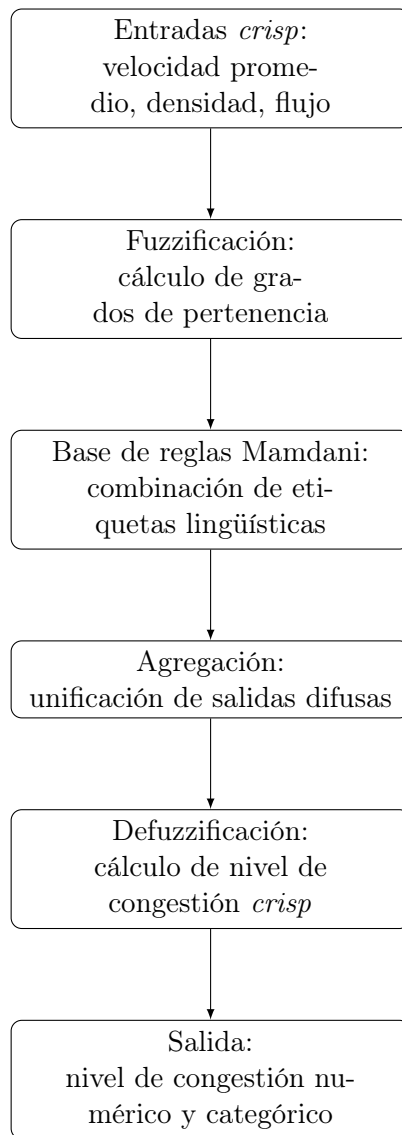


Figura 6.2: Etapas principales de un sistema de inferencia difusa tipo Mamdani aplicado a congestión vehicular [25, 48].

las entradas para el sistema difuso, ejecuta la evaluación Mamdani sobre cada uno y genera un conjunto de salidas que se utilizarán como insumo para la etapa de optimización PSO. De esta forma, la lógica difusa actúa como módulo de evaluación de desempeño que transforma mediciones de tráfico en un índice de congestión interpretable, que el optimizador buscará reducir.

La Tabla 6.2 presenta la base de reglas completa del sistema de inferencia Mamdani implementado en traffic-sync. Las 27 reglas cubren todas las combinaciones posibles de los tres términos lingüísticos de cada variable de entrada.

Cuadro 6.2: Base de reglas completa del sistema de inferencia difusa Mamdani (27 reglas)

Vehículos/min	Velocidad	Densidad	Congestión
Bajo	Baja	Baja	Leve
Bajo	Baja	Media	Leve
Bajo	Baja	Alta	Severa
Bajo	Media	Baja	Ninguna
Bajo	Media	Media	Leve
Bajo	Media	Alta	Leve
Bajo	Alta	Baja	Ninguna
Bajo	Alta	Media	Ninguna
Bajo	Alta	Alta	Leve
Medio	Baja	Baja	Leve
Medio	Baja	Media	Severa
Medio	Baja	Alta	Severa
Medio	Media	Baja	Leve
Medio	Media	Media	Leve
Medio	Media	Alta	Severa
Medio	Alta	Baja	Ninguna
Medio	Alta	Media	Leve
Medio	Alta	Alta	Leve
Alto	Baja	Baja	Severa
Alto	Baja	Media	Severa
Alto	Baja	Alta	Severa
Alto	Media	Baja	Leve
Alto	Media	Media	Severa
Alto	Media	Alta	Severa
Alto	Alta	Baja	Leve
Alto	Alta	Media	Leve
Alto	Alta	Alta	Severa

6.2. Optimización de Enjambre de Partículas (PSO)

Optimización de Enjambre de Partículas (PSO) es un algoritmo de optimización metaheurística inspirado en el comportamiento colectivo de bandadas de aves y bancos de peces, en el que un conjunto de partículas explora el espacio de búsqueda guiado por su mejor experiencia individual y la mejor experiencia global del grupo [7, 49]. Cada partícula representa una

solución candidata (en este caso, una posible asignación de tiempo de verde para un grupo de semáforos) y actualiza su posición combinando tres componentes: inercia, atracción hacia su mejor posición personal y atracción hacia la mejor posición global conocida, ponderadas por parámetros que controlan el equilibrio entre exploración y explotación [50, 51].

La Figura 6.3 ilustra esquemáticamente el concepto del enjambre PSO en un espacio de búsqueda bidimensional, mostrando cómo las partículas se mueven influenciadas por su mejor experiencia personal (p_i) y la mejor experiencia global del grupo (g).

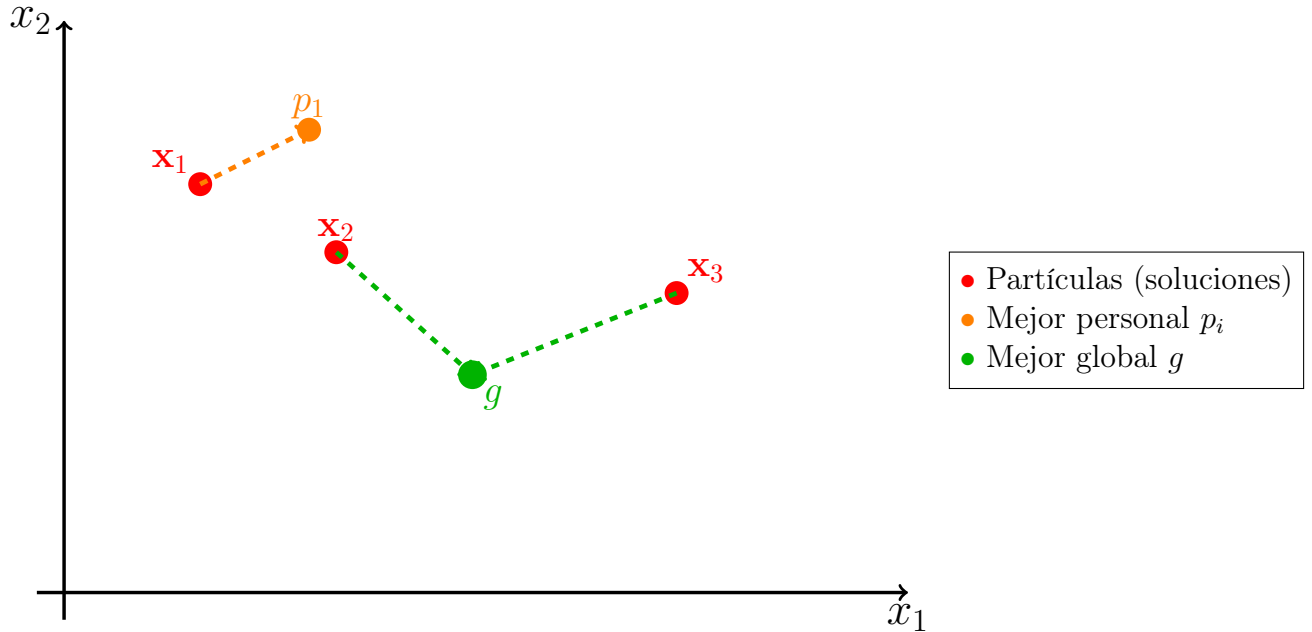


Figura 6.3: Esquema conceptual del enjambre PSO en un espacio de búsqueda bidimensional [7].

En el módulo traffic-sync, la lógica de PSO se implementa en el módulo de optimización por enjambre de partículas, que incluye el módulo de función de aptitud y el módulo de optimización. El módulo de optimización define la función principal de PSO, que recibe información agregada de tráfico por grupo (clúster) y aplica PSO para determinar el tiempo de verde óptimo por grupo. Cada partícula codifica un tiempo de verde propuesto para el ciclo semafórico, inicializado alrededor de un valor base calculado mediante fórmulas clásicas de ingeniería de tráfico (función de cálculo de tiempo de verde en el módulo de aptitud), y restringido por límites mínimos y máximos coherentes con la normativa [17, 2]. El algoritmo utiliza configuraciones típicas de PSO (número de partículas, peso de inercia, componentes cognitiva y social, límites de velocidad) y añade mecanismos prácticos como parada temprana y reinicios parciales para evitar estancamiento en óptimos locales pobres.

La Tabla 6.3 especifica la configuración completa del algoritmo PSO implementado en traffic-sync.

La función de aptitud evalúa cada propuesta de tiempo de verde t_v combinando las métricas del clúster con el sistema difuso Mamdani [1, 3]. Dado un clúster con valores medios de vehículos

Cuadro 6.3: Parámetros de configuración del algoritmo PSO en traffic-sync

Parámetro	Símbolo	Valor
Número de partículas	N_p	30
Iteraciones máximas	T_{max}	100
Factor de inercia	w	0,8
Componente cognitiva	c_1	1,5
Componente social	c_2	1,5
Paciencia (parada temprana)	—	10 iteraciones
Umbral de mejora	ε	1×10^{-3}
Tiempo de verde mínimo	t_{min}	10 s
Ciclo semafórico total	T_c	90 s
Espacio de búsqueda	t_v	[10, 70] s

por minuto (VPM), velocidad (v) y densidad (ρ), se computan los parámetros ajustados según la asignación t_v :

$$VPM_{adj} = VPM \cdot \frac{t_v}{T_c} \quad (6.1)$$

$$v_{adj} = v \cdot (1 + 0,01 \cdot (t_v - t_{min})) \quad (6.2)$$

$$\rho_{adj} = \rho \cdot \frac{T_c - t_v}{T_c} \quad (6.3)$$

donde $T_c = 90$ s es el ciclo semafórico total y $t_{min} = 10$ s es el tiempo mínimo de verde. La aptitud de la partícula se define como el nivel de congestión difusa re-evaluado con los parámetros ajustados:

$$f(t_v) = \text{Mamdani}(VPM_{adj}, v_{adj}, \rho_{adj}) \quad (6.4)$$

El PSO minimiza $f(t_v)$ sobre $t_v \in [t_{min}, T_c - 20]$. La interpretación es directa: un mayor tiempo de verde permite que más vehículos atraviesen la intersección por ciclo (aumenta VPM_{adj}), mejora levemente la velocidad media y reduce la densidad acumulada durante la fase roja; el sistema Mamdani re-evalúa esas condiciones ajustadas y devuelve un índice de congestión menor. De este modo, el PSO busca explícitamente reducir el índice de congestión proporcionado por el módulo difuso, sin requerir una función analítica cerrada.

En el pipeline de traffic-sync, orquestado por la función principal del servicio web, el flujo es el siguiente: (i) el servicio recibe un lote de sensores en la API `/evaluate`; (ii) las mediciones se pasan al módulo difuso para obtener un nivel de congestión por sensor; (iii) se agrupan sensores con características similares (agrupamiento jerárquico), produciendo un conjunto de grupos de tráfico; (iv) se construye un conjunto de datos por grupo con métricas agregadas (por ejemplo, vehículos por minuto, velocidad media, densidad media, categoría de congestión); y (v) se aplica la función de optimización PSO sobre estos grupos para obtener tiempos de verde optimizados por grupo. La respuesta de la API se construye como un lote de optimizaciones, donde cada

entrada incluye el tiempo de verde recomendado, el tiempo de rojo derivado, el nivel de congestión original y optimizado, y las categorías lingüísticas correspondientes, preparados para su consumo posterior por el módulo de control de semáforos y por el módulo de almacenamiento.

En conjunto, la integración entre lógica difusa Mamdani y PSO en traffic-sync permite que la optimización de tiempos semafóricos se base en una medida de congestión que captura adecuadamente la percepción y el comportamiento del tráfico urbano, mientras que el metaheurístico proporciona un mecanismo flexible para ajustar los parámetros de control en escenarios donde la función objetivo no es derivable y debe evaluarse a través de simulación o reglas de inferencia [27, 52].

La Figura 6.4 ilustra la arquitectura general del módulo traffic-sync, mostrando la interacción entre los componentes de lógica difusa y PSO, así como el flujo de datos desde la recepción de métricas de tráfico hasta la generación de configuraciones optimizadas de tiempos semafóricos.

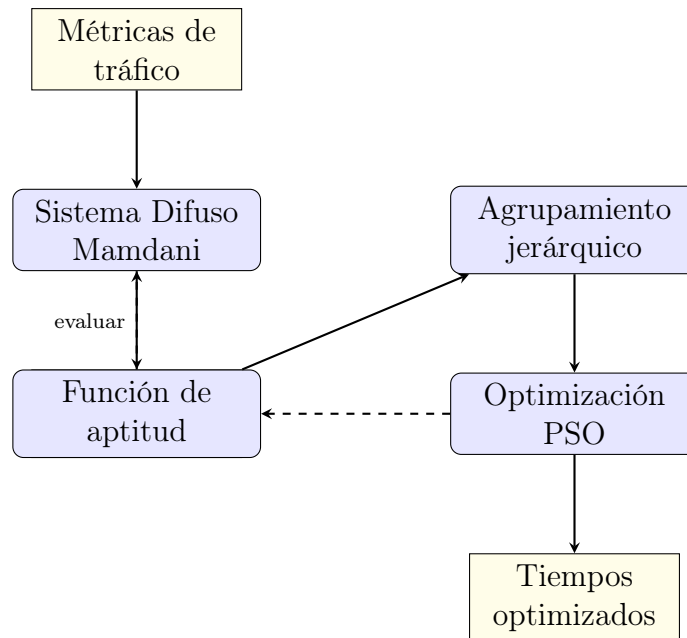


Figura 6.4: Arquitectura del módulo traffic-sync integrando lógica difusa Mamdani y optimización PSO.

6.3. Ejemplos de Ciclos de Optimización

La Tabla 6.4 presenta cuatro ejecuciones reales del endpoint `POST /evaluate` de traffic-sync, con condiciones de tráfico que cubren el rango de operación del sistema: desde flujo libre hasta congestión severa. En cada caso se muestran las métricas de entrada observadas, el nivel de congestión evaluado por el sistema Mamdani antes de la optimización, el tiempo de verde asignado por PSO y el nivel de congestión estimado con ese tiempo de verde.

En los escenarios de flujo libre y moderado el optimizador reduce la congestión estimada a nivel *none* (valor 1), asignando tiempos de verde acordes con la demanda observada. En

Cuadro 6.4: Resultados reales del módulo traffic-sync ante cuatro escenarios de tráfico.

Entrada (observada)				Salida PSO			
Escenario	VPM	Vel. (km/h)	Dens.	Cong. orig. (0–10)	Verde (s)	Cong. optim. (0–10)	Categ. orig. → optim.
Flujo libre	8	48,0	0,12	1	14	1	<i>none</i> → <i>none</i>
Flujo moderado	18	22,0	0,39	4	64	1	<i>mild</i> → <i>none</i>
Flujo alto	28	32,0	0,52	5	19	1	<i>mild</i> → <i>none</i>
Congestión severa	42	14,0	0,81	8	47	4	<i>severe</i> → <i>mild</i>

el escenario de congestión severa (42 vehículos/min, 14 km/h, densidad 0,81), PSO no logra eliminar la congestión pero sí la reduce de categoría *severe* a *mild* (de 8 a 4 en la escala de 0 a 10), asignando un tiempo de verde de 47 s frente a los 14 s del caso de flujo libre. Este comportamiento es coherente con el diseño del sistema: la función de aptitud proyecta el efecto del tiempo de verde candidato sobre las variables macroscópicas de tráfico y re-evalúa la congestión con el sistema difuso, de modo que tiempos de verde más largos reducen la densidad y el tiempo de circulación estimados, desplazando la salida del sistema Mamdani hacia categorías de menor congestión.

CAPÍTULO 7

CONTROL Y ORQUESTACIÓN DEL SISTEMA

El módulo `traffic-control` actúa como orquestador central del sistema distribuido de gestión de tráfico: recibe observaciones vehiculares desde simulaciones o despliegues reales, valida y registra los datos, coordina el envío de información hacia los servicios de optimización (`traffic-sync`) y almacenamiento distribuido (`traffic-storage`), y expone una API REST unificada para clientes externos. Su objetivo es desacoplar los productores de datos (por ejemplo, `traffic-sim`) de los módulos especializados de optimización y persistencia, garantizando un flujo coherente y trazable de información a través del sistema.

7.1. Arquitectura del Módulo

La estructura de `traffic-control` se organiza en capas bien definidas: configuración, API, modelos de datos, servicios de dominio y utilidades de soporte. El servicio web principal (basado en FastAPI) implementa el punto de entrada `/process`, que procesa observaciones vehiculares completas, y puntos de entrada auxiliares como `/healthcheck` y operaciones sobre metadatos. El módulo de configuración centraliza parámetros del sistema, como las URLs de los servicios de almacenamiento y optimización, la cadena de conexión a la base de datos y los niveles de logging. El módulo de configuración de logs define la estructura de registro para toda la aplicación.

La capa de modelos incluye esquemas de validación para estructurar peticiones y respuestas (esquemas de datos, optimización y descarga) y modelos de respuesta estandarizados. El módulo de validación encapsula la validación semántica de los datos: verifica versión de la carga útil, formato de marcas de tiempo, tipo de mensaje (datos u optimización), límites de tamaño de lote (1–10 sensores) e integridad de campos como identificador de semáforo, métricas y estadísticas vehiculares. Cualquier inconsistencia o violación de estos contratos se transforma

en una respuesta de error coherente mediante el manejador centralizado de errores.

La persistencia principal del sistema se realiza mediante IPFS y BlockDAG a través del módulo de almacenamiento, que garantiza almacenamiento distribuido, inmutabilidad y trazabilidad verificable. Complementariamente, el módulo de base de datos gestiona un índice auxiliar de metadatos en una base de datos SQL local, con el punto de entrada a la conexión de base de datos y el modelo de metadatos definiendo el esquema de almacenamiento: cada registro guarda, como mínimo, el identificador del semáforo, la marca de tiempo, el tipo de dato (por ejemplo, observación de tráfico u optimización) y referencias a los identificadores de contenido (CIDs y resúmenes criptográficos de transacción) almacenados en el módulo de almacenamiento. El servicio de base de datos proporciona operaciones de alto nivel para crear y consultar este índice local, que se exponen a través de puntos de entrada de metadatos (por ejemplo, `/metadata/traffic-light/{id}`, `/metadata/recent`, `/metadata/stats`) para facilitar consultas rápidas, mientras que los datos completos permanecen en el almacenamiento distribuido.

La Figura 7.1 ilustra la arquitectura del módulo traffic-control, mostrando las capas de API, validación, servicios de dominio y persistencia, así como la integración con los módulos especializados del sistema.

7.2. Flujo de Procesamiento

El punto de entrada `POST /process` es el principal para observaciones de tráfico, tanto provenientes de traffic-sim como de otros clientes. El flujo de procesamiento sigue una secuencia de pasos bien definida:

1. **Recepción y validación:** la API recibe una carga útil en formato unificado (versión 2.0), que puede contener uno o varios sensores asociados a un mismo identificador de semáforo. El módulo de validación verifica que el mensaje cumpla con la estructura esperada (campos obligatorios en métricas y estadísticas vehiculares, rango de tamaño de lote, formatos de fecha, tipo de mensaje), generando errores explícitos en caso de inconsistencias.
2. **Preprocesamiento de datos:** el servicio de procesamiento de datos puede aplicar transformaciones ligeras sobre los datos (normalización de campos, ajuste de formatos, enriquecimiento con información temporal) antes de enviarlos a los módulos externos. Esta capa asegura que, aunque distintos productores tengan pequeñas variaciones, el sistema trabaje con un formato interno coherente.
3. **Almacenamiento distribuido principal:** el proxy de almacenamiento construye una petición hacia la API del módulo de almacenamiento, que implementa el almacenamiento principal del sistema mediante IPFS y BlockDAG. Los datos completos se almacenan de forma distribuida e inmutable, recibiendo de vuelta identificadores de contenido (CIDs de IPFS) y resúmenes criptográficos de transacción en BlockDAG. Este es el mecanismo de

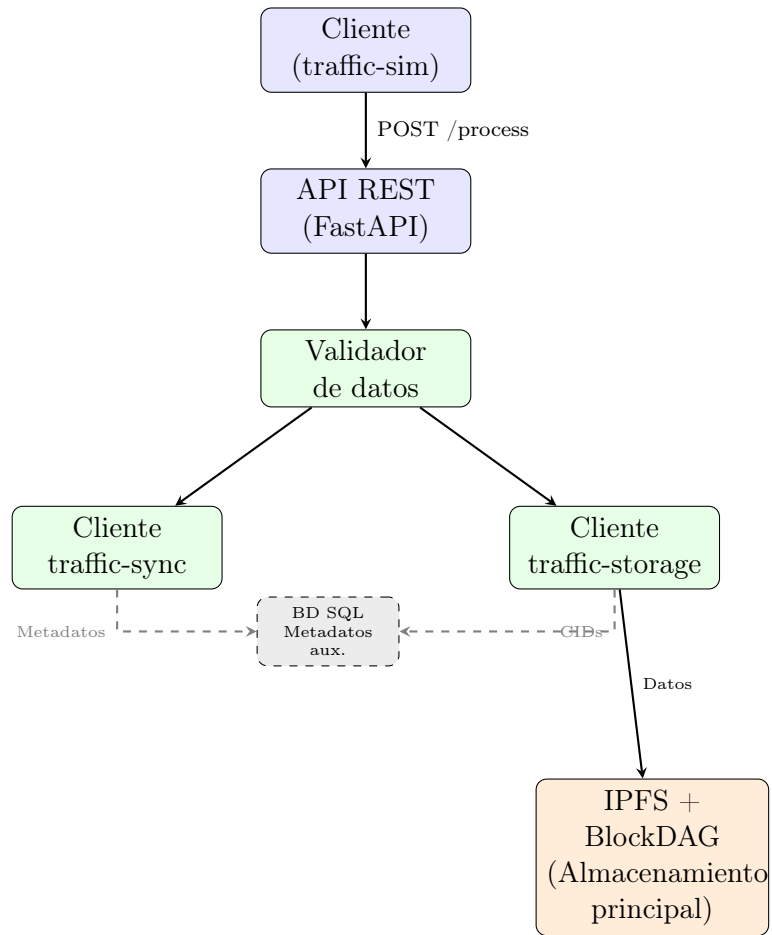


Figura 7.1: Arquitectura del módulo traffic-control mostrando el flujo de orquestación. El almacenamiento principal se realiza mediante IPFS y BlockDAG a través de traffic-storage, mientras que la base de datos SQL se utiliza únicamente como índice auxiliar de metadatos.

persistencia central del sistema, garantizando trazabilidad, integridad e inmutabilidad de los registros históricos.

4. **Índice auxiliar de metadatos:** mediante el servicio de base de datos, se registra en una base de datos SQL local un resumen del evento (identificador de semáforo, marca de tiempo, tipo, estado del procesamiento, CIDs y resúmenes criptográficos de transacción), lo que permite consultas rápidas sobre qué datos fueron recibidos, cuándo y con qué resultado. Esta base de datos actúa únicamente como índice auxiliar para facilitar consultas locales, complementando los registros inmutables almacenados en IPFS y BlockDAG gestionados por el módulo de almacenamiento.
5. **Interacción con el módulo de optimización:** el proxy de sincronización envía los datos de tráfico al módulo de optimización mediante el punto de entrada `/evaluate`, en formato compatible con el sistema difuso y el optimizador PSO. Para lotes de sensores, la función de envío por lotes reempaqueta el lote en el formato esperado por el módulo de optimización (incluyendo el campo de identificador de semáforo de referencia) y recibe un conjunto de optimizaciones, una por grupo detectado.
6. **Respuesta al cliente:** finalmente, el servicio de procesamiento compone una respuesta estandarizada utilizando los modelos de respuesta definidos, indicando el estado del procesamiento (por ejemplo, éxito o error), un mensaje descriptivo y, cuando corresponda, datos derivados como tiempos de verde optimizados e impacto estimado sobre la congestión. Esta respuesta se devuelve al cliente original (por ejemplo, el módulo de simulación), que puede aplicar las decisiones de control correspondientes.

Durante todo el flujo, el manejador centralizado de errores proporciona mecanismos para capturar y clasificar errores en distintas categorías (validación, almacenamiento, sincronización, base de datos, errores genéricos), generando respuestas HTTP adecuadas y registros de log con contexto detallado para facilitar la depuración.

7.3. Interacción con los Módulos

Desde el punto de vista del sistema global, traffic-control se posiciona como el módulo de orquestación que articula la interacción entre simulación, optimización y almacenamiento distribuido:

- **Relación con traffic-sim:** el módulo de simulación basado en SUMO detecta cuellos de botella y, cuando identifica una situación crítica en una intersección, construye una carga útil con métricas agregadas de tráfico y lo envía a traffic-control mediante el punto de entrada `/process`. De este modo, traffic-sim no necesita conocer detalles sobre traffic-sync ni traffic-storage; sólo interactúa con un servicio central que gestiona validación, almacenamiento y consulta de optimizaciones.

La Figura 7.2 muestra un diagrama de secuencia detallado del flujo completo de procesamiento en traffic-control, ilustrando las interacciones entre todos los componentes del sistema desde la recepción de datos hasta la persistencia distribuida.

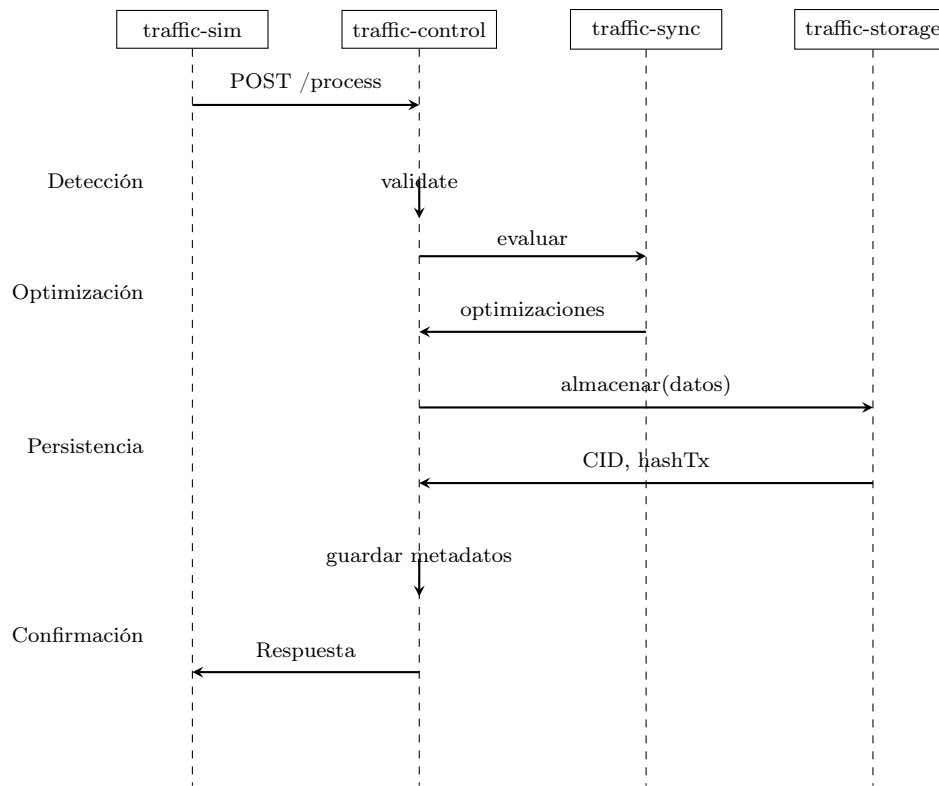


Figura 7.2: Diagrama de secuencia del flujo completo de procesamiento en el sistema distribuido.

- **Relación con el módulo de optimización:** traffic-control actúa como cliente del módulo de optimización a través del proxy de sincronización. Para cada observación (o lote) recibida, reenvía los datos relevantes al módulo de optimización, que ejecuta el pipeline de lógica difusa y PSO para producir tiempos de verde optimizados y categorías de congestión. La respuesta del módulo de optimización se integra en la respuesta de traffic-control, permitiendo que los consumidores (por ejemplo, el módulo de simulación) apliquen las configuraciones recomendadas.
- **Relación con el módulo de almacenamiento:** mediante el proxy de almacenamiento, traffic-control sube y recupera datos hacia/desde el módulo de almacenamiento, que a su vez utiliza IPFS, blockchain y BlockDAG como backend de almacenamiento y registro inmutable. traffic-control registra en su base de datos los identificadores de contenido (por ejemplo, CIDs) devueltos por el módulo de almacenamiento, de modo que las consultas de metadatos pueden resolverse localmente y, cuando es necesario, seguir la cadena de referencias hasta los datos almacenados en el sistema distribuido.

Esta arquitectura modular permite que cada componente se especialice en su responsabilidad principal: traffic-sim en la generación de escenarios y detección de cuellos de botella; traffic-sync

en la evaluación de congestión y optimización de tiempos semafóricos mediante lógica difusa y PSO; traffic-storage en la persistencia verificable de datos; y traffic-control en la validación, coordinación y exposición de un punto de entrada unificado al sistema. En conjunto, estos módulos forman una plataforma escalable y descentralizada para la gestión inteligente de tráfico urbano.

7.4. Contrato de la API REST

La Tabla 7.1 describe el contrato público de traffic-control: los endpoints disponibles, el método HTTP, los parámetros o cuerpo esperado y el código de respuesta en el caso exitoso.

Cuadro 7.1: Endpoints de la API REST de traffic-control.

Método	Ruta	Entrada / Parámetros	HTTP
GET	/healthcheck	—	200
POST	/process	JSON TrafficData (versión 2.0; 1–10 sensores)	200
GET	/metadata/traffic-light/{id}	?limit=100 (int)	200
GET	/metadata/type/{type}	type: data optimization; ?limit=100	200
GET	/metadata/recent	?limit=50 (int)	200
GET	/metadata/stats	—	200
DELETE	/metadata/traffic-light/{id}	—	200

El endpoint `POST /process` es el punto de entrada principal. Acepta un JSON con los campos `version` (cadena semver), `type` ("data"), `timestamp` (ISO 8601 UTC), `traffic_light_id` (solo dígitos) y `sensors` (lista de 1 a 10 elementos con `metrics` y `vehicle_stats`). En caso de éxito, la respuesta contiene el estado del procesamiento, los tiempos de verde optimizados devueltos por traffic-sync y los identificadores de contenido (CID) y hash de transacción devueltos por traffic-storage. Los errores de validación retornan 422 (Unprocessable Entity) con detalle de campo; los errores de servicio externo retornan 503.

Los módulos internos exponen sus propios endpoints de uso exclusivo del orquestador: traffic-sync ofrece `POST /evaluate` (lista de observaciones de sensores, retorna tiempos de verde y nivel de congestión por grupo), y traffic-storage ofrece `POST /upload/` (sube datos a IPFS y registra el CID en BlockDAG, retorna CID y hash de transacción) y `POST /download/` (recupera datos de IPFS a partir de un CID). Estos endpoints no forman parte de la interfaz pública del sistema y no son accedidos directamente por traffic-sim.

CAPÍTULO 8

EVALUACIÓN EXPERIMENTAL DEL SISTEMA

Los capítulos anteriores han descrito la arquitectura, los fundamentos teóricos y la implementación de cada módulo del sistema propuesto. El presente capítulo tiene como objetivo demostrar empíricamente que el sistema cumple con los requisitos de rendimiento, integridad y eficiencia planteados para su aplicación en entornos de gestión de tráfico urbano. Para ello, se presentan los resultados de experimentos controlados ejecutados sobre los dos módulos que constituyen el núcleo computacional del sistema: `traffic-sync`, responsable del análisis de congestión y la optimización semafórica, y `traffic-storage`, encargado de la persistencia verificable y descentralizada de datos de tráfico.

Las evaluaciones persiguen cuatro objetivos específicos. En primer lugar, se estudia la escalabilidad del módulo de análisis y optimización, analizando cómo evolucionan el tiempo de procesamiento y el consumo de recursos del sistema conforme aumenta el número de sensores simulados que operan simultáneamente. En segundo lugar, se mide la eficiencia del subsistema de almacenamiento distribuido, cuantificando las latencias extremo a extremo de las operaciones de escritura y lectura sobre la capa IPFS–BlockDAG bajo distintas condiciones de carga vehicular. En tercer lugar, se verifica la integridad de los datos almacenados mediante verificación criptográfica, comprobando que los registros recuperados desde infraestructuras descentralizadas conservan fielmente el contenido original. En cuarto lugar, se contextualiza el desempeño obtenido mediante una comparación con tecnologías alternativas de almacenamiento distribuido (particularmente con redes blockchain basadas en Ethereum) con el fin de evidenciar las ventajas operativas de la arquitectura propuesta para aplicaciones de monitoreo urbano de alta frecuencia.

Todos los experimentos se ejecutaron en hardware de consumo estándar, sin requerir infraestructura especializada, lo que pone de manifiesto la viabilidad del sistema para despliegues en entornos municipales con recursos computacionales limitados.

8.1. Evaluación del módulo traffic-sync

El módulo traffic-sync constituye el motor analítico del sistema: recibe métricas de tráfico, clasifica el estado de congestión mediante un sistema de inferencia difusa tipo Mamdani y calcula las configuraciones óptimas de temporización semafórica mediante el algoritmo PSO. Si bien en la operación actual el módulo procesa un sensor a la vez, la arquitectura fue diseñada desde el inicio para soportar la evaluación conjunta de múltiples sensores agrupados por zonas urbanas. Con el fin de anticipar este escenario de uso futuro, en el que los clústeres de semáforos deberán analizarse en paralelo para lograr una dispersión efectiva del tráfico, se realizaron pruebas de escalabilidad simulando entre 250 y 2000 sensores emitiendo datos de forma simultánea.

8.1.1. Entorno y Protocolo de Prueba

Los experimentos se realizaron en dos máquinas de cómputo de propósito general, sin hardware acelerador y sin soporte de red especializada, reflejando las condiciones típicas de un entorno de desarrollo o despliegue municipal de bajo presupuesto. La evaluación del módulo traffic-sync se llevó a cabo en una laptop equipada con un procesador AMD Ryzen 5 5500U (6 núcleos, 12 hilos, frecuencia máxima de 4,05 GHz), 7,1 GB de memoria RAM disponible y sistema operativo Ubuntu 24.04 LTS. La evaluación del módulo traffic-storage, que requería conectividad real con servicios distribuidos externos, se realizó en un equipo con procesador AMD Ryzen 7 3700U (4 núcleos, 8 hilos, frecuencia base de 2,3 GHz), 9,6 GB de memoria RAM total (5,9 GB disponibles durante las pruebas), almacenamiento SSD NVMe de 233 GB y el mismo sistema operativo Ubuntu 24.04.2 LTS. Ambas configuraciones corresponden a hardware de consumo estándar disponible comercialmente, hecho que refuerza la premisa de diseño del sistema: su operatividad no depende de servidores de alta gama ni de plataformas en la nube.

Pila de Software

El sistema se ejecutó sobre Python 3 utilizando FastAPI como framework para las APIs RESTful de cada módulo. La interacción con la red BlockDAG se gestionó mediante la biblioteca Web3.py, mientras que el procesamiento de datos de simulación se apoyó en las herramientas provistas por SUMO¹ a través del protocolo TraCI. La capa de almacenamiento en IPFS se implementó utilizando Pinata² como servicio de nodo gestionado, con autenticación mediante tokens JWT y un gateway personalizado para las operaciones de descarga. La validación de esquemas de datos en tiempo de ejecución se realizó mediante Pydantic, que garantiza la consistencia estructural de los mensajes JSON intercambiados entre módulos.

¹<https://github.com/eclipse-sumo/sumo>

²<https://pinata.cloud>

Infraestructura de Red Descentralizada

Las transacciones de almacenamiento se enviaron a la red *Primordial TestNet*, una red BlockDAG de pruebas con identificador de cadena (Chain ID) 1043, accesible a través del nodo RPC público <https://rpc.primordial.bdagscan.com> [43]. El contrato inteligente TrafficStorage.sol, implementado en Solidity 0.8.19, se encuentra desplegado en la dirección de contrato 0xC3d520EBE9A9F52FC5E1519f

17F5a9A01d8ac68f de dicha red. El contrato expone dos funciones principales: `storeRecord`, que registra un nuevo CID de IPFS asociado a un identificador de semáforo, una marca de tiempo y un tipo de dato; y `getRecord`, que recupera el CID correspondiente a una consulta determinada. Ambas interactúan con un mapeo anidado indexado por `trafficLightId`, `timestamp` y `DataType`, cuyo valor almacenado es el CID correspondiente, garantizando la unicidad de cada registro y facilitando consultas directas sin recorridos costosos sobre la estructura de datos del contrato.

La Tabla 8.1 resume las especificaciones de ambos entornos de prueba para facilitar la reproducibilidad de los experimentos.

Cuadro 8.1: Especificaciones del entorno experimental

Parámetro	Entorno A (traffic-sync)	Entorno B (traffic-storage)
Procesador	AMD Ryzen 5 5500U	AMD Ryzen 7 3700U
Núcleos / Hilos	6 / 12	4 / 8
Frec. máxima	4,05 GHz	2,3 GHz (base)
RAM disponible	7,1 GB	5,9 GB
Almacenamiento	N/A	233 GB SSD NVMe
Sistema operativo	Ubuntu 24.04 LTS	Ubuntu 24.04.2 LTS
GPU	No utilizada	No utilizada
Red	Wi-Fi doméstica	Wi-Fi doméstica

Se definieron siete niveles de carga: 250, 500, 1000, 1250, 1500, 1750 y 2000 sensores simulados. Para cada nivel, se midieron tres métricas: el tiempo total de ejecución del ciclo de análisis y optimización (en segundos), el uso promedio de CPU durante el procesamiento (en porcentaje) y el uso de memoria RAM en el punto de mayor carga (en megabytes). Las pruebas se ejecutaron en el Entorno A (laptop AMD Ryzen 5 5500U, Ubuntu 24.04 LTS), sin soporte de GPU y bajo condiciones de red doméstica, reflejando un escenario de despliegue realista en hardware de bajo costo. Los datos de entrada correspondieron a observaciones sintéticas en el formato unificado versión 2.0, incluyendo métricas de velocidad promedio, densidad vehicular y número de vehículos por minuto.

8.1.2. Tiempo de Ejecución

La Figura 8.1 muestra la evolución del tiempo total de ejecución en función de la cantidad de sensores simulados. Los valores obtenidos, que van desde 15,87 s para 250 sensores hasta 67,59 s

para 2000 sensores, evidencian un crecimiento aproximadamente lineal con una pendiente de $\approx 0,034$ s por sensor adicional procesado. Este comportamiento lineal es altamente favorable: indica que el módulo no incurre en sobrecarga superlineal a medida que se incrementa la carga de trabajo, y que los tiempos de respuesta son predecibles y controlables mediante particionamiento horizontal de los sensores. Para la escala más exigente evaluada (2000 sensores), el ciclo completo de análisis y optimización se completa en 67,6 s, lo que representa una capacidad de $\approx 0,034$ s/sensor y confirma la viabilidad del módulo para escenarios de optimización por zonas a escala urbana real.

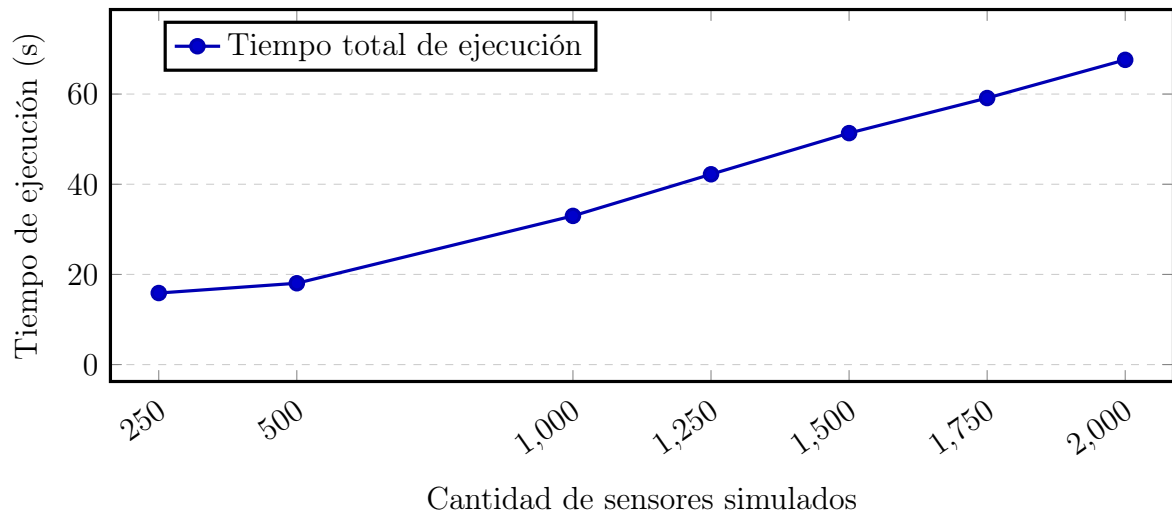


Figura 8.1: Tiempo de ejecución del módulo traffic-sync en función de la cantidad de sensores simulados. El crecimiento aproximadamente lineal ($\approx 0,034$ s/sensor) confirma la escalabilidad del módulo.

8.1.3. Uso de CPU y Memoria

La Figura 8.2 presenta de forma simultánea el uso promedio de CPU y el consumo de memoria RAM bajo cada nivel de carga. El comportamiento de ambas métricas exhibe tendencias en apariencia contrapuestas pero explicables por la naturaleza de la implementación.

El uso de CPU disminuye progresivamente desde el 72,3 % observado con 250 sensores hasta el 50,3 % con 2000 sensores. Esta reducción relativa se explica por la naturaleza de la medición: el porcentaje de CPU se calcula como la razón entre el tiempo de procesador activo y el tiempo total transcurrido (tiempo de pared). A medida que el volumen de sensores crece, el tiempo de pared aumenta de forma aproximadamente lineal, mientras que el tiempo de CPU activo por unidad de trabajo permanece prácticamente constante; el denominador crece más rápido que el numerador. Entre lotes sucesivos existen períodos de relativa inactividad (serialización de respuestas, despacho de peticiones HTTP, sobrecarga del intérprete Python) que incrementan el tiempo de pared sin contribuir proporcionalmente a los ciclos de procesador. Cabe señalar que tanto el sistema de inferencia difusa Mamdani como el optimizador PSO son operaciones mayoritariamente CPU-bound por unidad; la caída del porcentaje refleja un efecto de dilución

temporal, no una reducción del trabajo computacional efectivo por sensor.

El consumo de memoria RAM, por su parte, se mantiene estable en el rango de 198–219 MB en todos los escenarios evaluados, creciendo apenas 20,8 MB al pasar de 250 a 2000 sensores (+10,5 %). Esta estabilidad es un indicador clave de la viabilidad del módulo en dispositivos de recursos acotados, como unidades de cómputo embebido tipo Raspberry Pi, donde la memoria disponible suele ser el factor limitante más crítico.

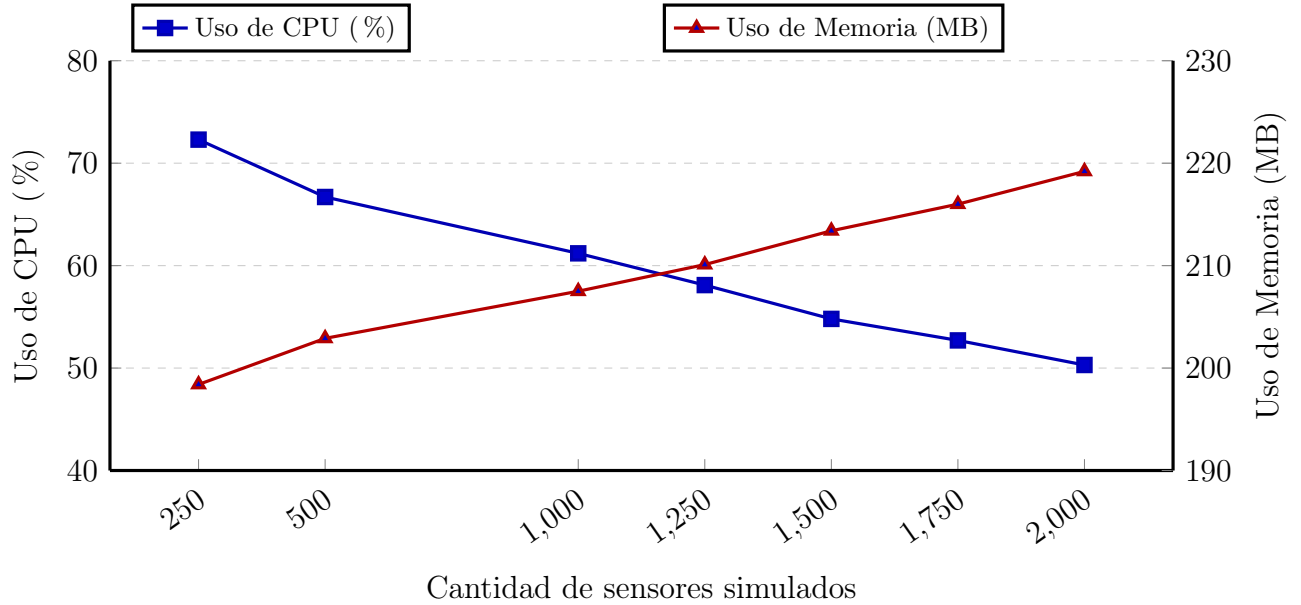


Figura 8.2: Uso promedio de CPU y consumo de memoria RAM del módulo traffic-sync bajo diferentes cargas de sensores simulados. El CPU disminuye conforme aumenta la concurrencia; la memoria permanece prácticamente estable en el rango 200–220 MB.

8.2. Evaluación del módulo traffic-storage

El módulo traffic-storage es la capa de persistencia verificable del sistema. Ante cada reporte de tráfico o resultado de optimización, el módulo sube el contenido en formato JSON a IPFS y registra el identificador de contenido (CID) resultante en el contrato inteligente de la red BlockDAG, vinculándolo de forma inmutable al semáforo de origen y la marca de tiempo correspondiente. La evaluación de este módulo se realizó en dos etapas complementarias: primero, una medición de métricas de transacción base que caracteriza la latencia, el costo y el consumo de gas del pipeline completo; y segundo, un protocolo de prueba por escenarios de tráfico vehicular simulado que valida el comportamiento del sistema bajo distintas densidades de datos.

8.2.1. Métricas de Transacción Base

Para caracterizar el desempeño del pipeline de almacenamiento en condiciones controladas, se ejecutaron múltiples transacciones reales de subida y descarga sobre un nodo IPFS local

(Kubo) con conexión directa a la red BlockDAG, midiendo cuatro indicadores: tiempo total de la operación extremo a extremo, tiempo de confirmación en la red BlockDAG, latencia de red y costo por transacción. La Tabla 8.2 resume los resultados obtenidos.

Cuadro 8.2: Métricas de rendimiento del módulo traffic-storage (IPFS + BlockDAG)

Métrica	Subida (upload)	Bajada (download)
Tiempo total (s)	$2,47 \pm 0,26$	$1,91 \pm 0,19$
Confirmación (s)	$1,12 \pm 0,14$	$0,98 \pm 0,11$
Latencia de red (ms)	184 ± 21	167 ± 19
Gas utilizado (upload)	36,232–95,624 [†]	—
Costo estimado (USD)	$< 0,00002$ – $< 0,00005$ [†]	—

[†] Rango entre datos de tráfico (36.232 unidades, $< \$0,00002$) y resultados de optimización (95.624 unidades, $< \$0,00005$); la descarga usa `getRecord`, función `view` sin costo on-chain. Detalle por tipo en Tabla 8.5.

Los tiempos de confirmación en la red BlockDAG, del orden de 1 segundo, y el costo por transacción, inferior a $\$5 \times 10^{-5}$ USD según el tipo de dato almacenado, confirman la viabilidad del sistema para monitoreo de alta frecuencia; la comparación con tecnologías alternativas se detalla en la Sección 8.3.

8.2.2. Pruebas con Escenarios de Tráfico Vehicular

Con el objeto de evaluar el comportamiento del módulo ante volúmenes de datos representativos de distintas condiciones operativas reales, se diseñó un protocolo de prueba basado en tres escenarios de densidad vehicular: bajo, con 30–50 vehículos activos y condiciones de tráfico fluido; medio, con 80–120 vehículos y tráfico normal en horario diurno; y alto, con 150–200 vehículos en situación de congestión o pico de demanda. Los payloads de cada escenario se generaron de forma sintética utilizando el formato unificado versión 2.0 del sistema, incluyendo métricas por semáforo como vehículos por minuto, velocidad promedio, tiempo de circulación, densidad y clasificación por tipo de vehículo (motocicletas, automóviles, autobuses y camiones). A diferencia de las pruebas de la Sección 8.2.1, este protocolo utilizó Pinata como servicio IPFS gestionado en la nube, lo que introduce latencia adicional por autenticación remota y transferencia vía gateway externo; esto explica que los tiempos de subida reportados aquí (6,7–7,3 s) sean superiores a los obtenidos con nodo local (2,47 s). Cada escenario se ejecutó mediante 5 corridas independientes para obtener estimaciones estadísticamente representativas de media y desviación estándar.

La Tabla 8.3 presenta los tiempos promedio de subida y descarga para cada escenario, considerando únicamente las corridas con respuesta HTTP 200, que son aquellas en las que se completó el ciclo completo de almacenamiento y recuperación en la red BlockDAG.

La Figura 8.3 ilustra gráficamente los tiempos promedio de cada operación con sus barras de error correspondientes a una desviación estándar. Puede observarse que el tiempo de subida crece levemente con la densidad vehicular, de 6,725 s en el escenario bajo a 7,290 s en el

Cuadro 8.3: Tiempos de subida y descarga por escenario de tráfico (5 corridas válidas por escenario)

Escenario	Corridas válidas	Subida (avg $\pm$ sd, s)	Descarga (avg $\pm$ sd, s)
Bajo	5	6,725 $\pm$ 1,035	1,981 $\pm$ 0,861
Medio	5	6,896 $\pm$ 0,963	1,750 $\pm$ 0,136
Alto	5	7,290 $\pm$ 0,787	1,767 $\pm$ 0,174

escenario alto, un incremento del 8,4 %, mientras que el tiempo de descarga permanece prácticamente estable en todos los escenarios, con variaciones inferiores a 0,25 s. Este comportamiento confirma que el módulo escala con el volumen de datos sin comprometer significativamente el rendimiento: el tamaño del payload influye de forma marginal en la latencia total, que está dominada principalmente por los tiempos de red y confirmación en la cadena.

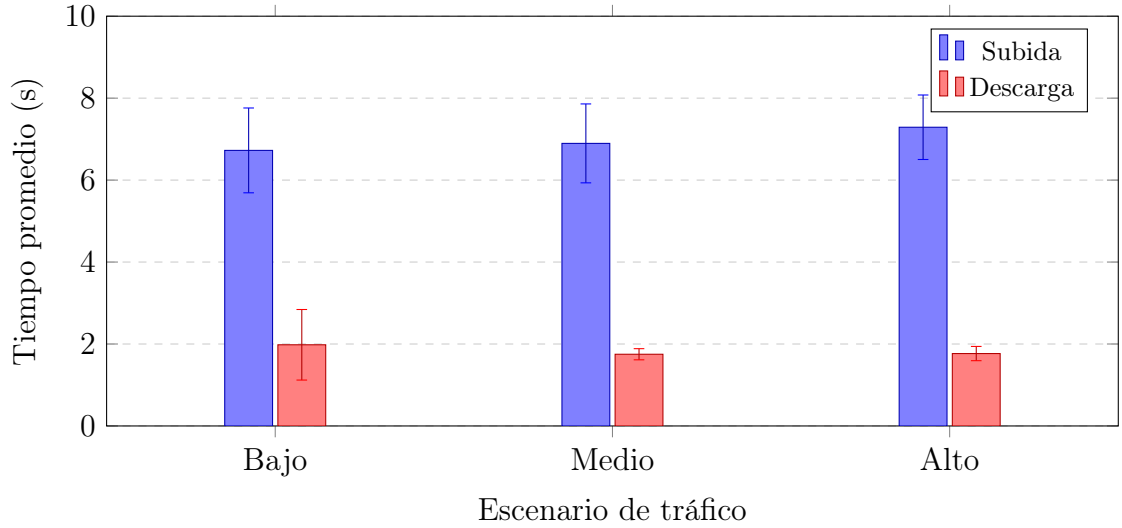


Figura 8.3: Tiempos promedio de subida y descarga del módulo traffic-storage por escenario de tráfico. Las barras de error representan ± 1 desviación estándar. El incremento de latencia de subida entre el escenario bajo y el alto es del 8,4 %.

8.2.3. Descomposición del Pipeline de Almacenamiento

El tiempo total de una operación de subida no es atómico: comprende varias etapas secuenciales con latencias propias. El análisis de los registros de ejecución detallados permite identificar las tres fases que componen el pipeline de almacenamiento. La primera fase es la subida a IPFS: la serialización del payload JSON, la autenticación con la API de Pinata y la transferencia del contenido hasta obtener el CID insume aproximadamente 1,2 s. La segunda fase es la preparación de la transacción BlockDAG: incluye la verificación del estado del nodo y el balance de la cuenta, el cálculo dinámico del precio de gas, la obtención del nonce actual y la firma criptográfica de la transacción; en conjunto, esta fase demanda aproximadamente 2,3 s. La tercera fase es la confirmación en la red: el tiempo que transcurre desde el envío de la transacción hasta que es incluida en un bloque minado y confirmada es de aproximadamente

3,5 s en las pruebas por escenarios. Este valor es superior al registrado en las pruebas base (1,12 s, Tabla 8.2), diferencia atribuible a las condiciones variables de la red BlockDAG pública entre sesiones de prueba realizadas en distintas fechas.

La Figura 8.4 muestra la composición proporcional de estas etapas para las operaciones de subida y descarga. En el caso de la descarga, el pipeline se reduce a dos fases: la consulta al contrato inteligente para recuperar el CID asociado a un semáforo y timestamp determinados (0,9 s), seguida de la descarga del contenido desde el gateway de IPFS usando dicho CID (0,7 s), con un tiempo total de 1,6 s.

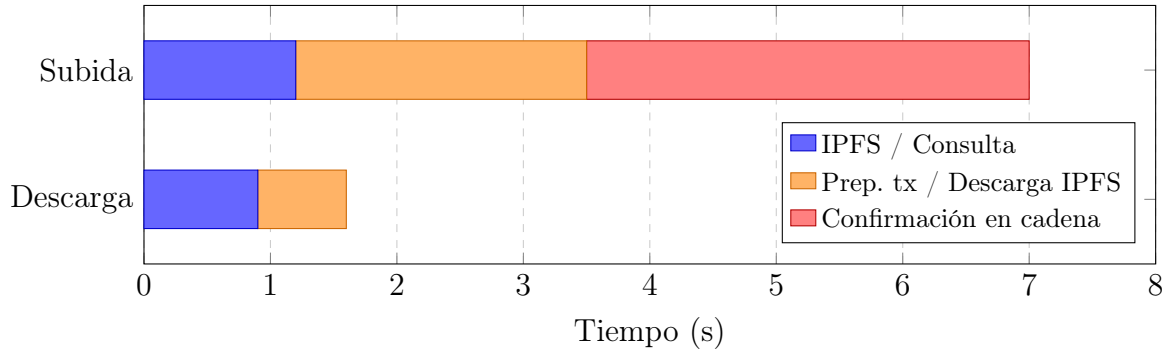


Figura 8.4: Desglose del tiempo de ejecución en las operaciones de subida y descarga del módulo traffic-storage. La confirmación en la red BlockDAG domina la latencia de subida ($\approx 50\%$), mientras que la descarga se completa en $\approx 1,6$ s sin necesidad de esperar confirmación.

8.2.4. Verificación de Integridad y Trazabilidad

Dado que el CID es el resumen criptográfico del contenido (véase la Sección 5.3), la recuperación exitosa del archivo original constituye por sí misma una prueba de integridad. Esta propiedad fue verificada de forma experimental durante las pruebas por escenarios: el contenido descargado en cada corrida produjo consistentemente el mismo resumen criptográfico, independientemente del momento de ejecución y del escenario de tráfico. Adicionalmente, el mismo CID fue recuperado en las cinco corridas de cada escenario, lo que confirma que el contrato inteligente almacena y retorna los metadatos de forma determinista y reproducible. La Tabla 8.4 presenta una muestra representativa de las transacciones registradas, incluyendo el CID de IPFS y el número de bloque en que fue confirmada cada operación.

Cuadro 8.4: Muestra de transacciones exitosas registradas durante las pruebas

Escenario	CID de IPFS (truncado)	Estado HTTP	Bloque
Bajo	Qmf9gGPs...G3AZox	200	confirmado
Medio	QmXkDdj8...zHSHR	200	confirmado
Alto	QmUK7Jxz...MDy8	200	confirmado

La trazabilidad de cada operación queda garantizada por el evento RecordStored emitido por el contrato inteligente en cada transacción exitosa, el cual registra de forma inmutable

el identificador del semáforo, la marca de tiempo, el tipo de dato y el CID en el historial de la red BlockDAG. Este mecanismo permite a cualquier auditor externo verificar de forma independiente que un dato determinado fue almacenado en un instante concreto, sin necesidad de confiar en el operador del sistema.

8.2.5. Consumo de Gas y Costos Operativos

El consumo de gas de las transacciones en la red BlockDAG varía según el tipo de dato almacenado. Las operaciones de almacenamiento de datos de tráfico sin procesar consumen aproximadamente 36.232 unidades de gas, mientras que las operaciones de almacenamiento de resultados de optimización, que incluyen un mayor volumen de campos calculados, consumen aproximadamente 95.624 unidades. La Tabla 8.5 resume el consumo y el costo estimado en dólares para cada tipo de operación.

Cuadro 8.5: Consumo de gas y costo estimado por tipo de operación de almacenamiento

Tipo de dato	Gas consumido (unidades)	Costo estimado (USD)
Datos de tráfico (Data)	36.232	< 0,00002
Optimizaciones (Optimization)	95.624	< 0,00005

Incluso para el tipo de operación más costosa, el almacenamiento de resultados de optimización con 95.624 unidades de gas, el costo por transacción se mantiene por debajo de \$0,00005 USD. Considerando una frecuencia de reporte típica de un ciclo cada 60 segundos por semáforo, el costo anual de almacenamiento on-chain para un semáforo individual sería inferior a \$0,88 USD según las condiciones observadas durante los experimentos. Sin embargo, esta estimación debe interpretarse con cautela: los experimentos se realizaron sobre una red BlockDAG pública de pruebas (testnet), cuyo token no posee valor de mercado real. Los costos en una red de producción (mainnet) dependerán del precio del token en el momento del despliegue y de la demanda de la red, por lo que este valor constituye únicamente un orden de magnitud comparativo, no una proyección de costo operativo real.

8.3. Comparación con Tecnologías Alternativas

Los resultados experimentales adquieren mayor significado cuando se los sitúa en relación con las alternativas tecnológicas disponibles para el almacenamiento distribuido con trazabilidad. Esta sección compara la solución adoptada, IPFS combinado con una red BlockDAG, frente a la plataforma blockchain de propósito general más utilizada, Ethereum, tanto desde la perspectiva teórica apoyada en literatura existente como desde los valores medidos experimentalmente.

8.3.1. BlockDAG frente a Ethereum

Las características de ambas plataformas se describieron en el Capítulo 5; los resultados medidos en este trabajo, con confirmación en ≈ 1 s y costos inferiores a $\$1,2 \times 10^{-5}$ USD por transacción, ratifican las ventajas teóricas de BlockDAG en condiciones de uso real sobre la red Primordial TestNet. La Tabla 8.6 sintetiza la comparación desde la perspectiva del monitoreo urbano de tráfico.

Cuadro 8.6: Comparación técnica entre Ethereum y BlockDAG en el contexto de monitoreo urbano de tráfico. Valores de BlockDAG medidos en Primordial TestNet.

Criterio	Ethereum	BlockDAG
Latencia de confirmación	15–60 s [53, 42]	≈ 1 s (medido)
Costo por transacción	$\$0,50$ – $5,00$ USD [54]	$< \$0,00002$ USD (medido)
Throughput (TPS)	15–30 [41]	> 1.000 [9]
Modelo de consenso	Proof-of-Stake	PoW asíncrono
Estructura de datos	Cadena lineal	Grafo acíclico
Contratos inteligentes	Maduro (EVM, Solidity)	Disponibile (EVM-compatible)
Escalabilidad	Limitada	Alta
Compatibilidad edge/IoT	Baja (nodos pesados)	Alta (nodos ligeros)

8.3.2. Reducción de Latencia respecto a Plataformas Centralizadas

Más allá de la comparación entre plataformas descentralizadas, la arquitectura propuesta también supera a las soluciones IoT-cloud centralizadas convencionales en términos de latencia de persistencia verificable. Los sistemas centralizados típicos introducen latencias de confirmación de escritura del orden de 3–6 s cuando incluyen mecanismos de integridad (firmas digitales, logs de auditoría) [32, 53], mientras que la confirmación BlockDAG obtenida en este trabajo alcanzó ≈ 1 s. Esto representa una reducción de latencia superior al 60 % respecto a dichos sistemas, con la ventaja adicional de que la inmutabilidad está garantizada por el protocolo de consenso distribuido, y no por la confianza en un operador central, y el costo operativo es órdenes de magnitud inferior.

Este conjunto de propiedades, latencia baja, costo insignificante, inmutabilidad criptográfica e independencia de infraestructura centralizada, posiciona a la combinación IPFS+BlockDAG como una alternativa técnicamente superior para aplicaciones de monitoreo urbano con alta frecuencia de reporte y requisitos de trazabilidad auditables.

8.4. Discusión

Los resultados obtenidos permiten evaluar el sistema de forma integral, identificando sus fortalezas, sus limitaciones actuales y las líneas de trabajo que podrían potenciar su desempeño en escenarios de despliegue real.

8.4.1. Viabilidad del Sistema

Los resultados de ambos módulos confirman la viabilidad del sistema para entornos con presupuesto limitado, como ciudades intermedias de la región. El módulo traffic-sync escala linealmente y opera dentro de la memoria disponible en hardware embebido de bajo costo, lo que lo hace apto para despliegues sobre unidades Raspberry Pi en campo. En el módulo traffic-storage, la latencia de almacenamiento (6,7–7,3 s de subida) es aceptable para el modelo de reporte periódico del sistema, que opera sobre datos agregados de ciclos completados, no sobre control en tiempo estricto; y la descarga ($\approx 1,75$ s) es conveniente para flujos de auditoría y consulta. El cuello de botella identificado es el tiempo de confirmación en la red BlockDAG ($\approx 3,5$ s en las pruebas por escenarios; 1,12 s en las pruebas base con nodo local), cuyo impacto podría reducirse mediante la optimización de la gestión de nonces y la precarga del estado de gas antes del ciclo de almacenamiento, sin modificar la infraestructura de red.

8.4.2. Desafíos Identificados y Limitaciones

Durante la ejecución de las pruebas se identificaron tres desafíos técnicos que deben tenerse en cuenta en un despliegue de producción.

El primero es la dependencia de servicios IPFS gestionados: las pruebas utilizaron Pinata como nodo IPFS de terceros, lo que introduce una dependencia externa que podría comprometer la disponibilidad del sistema si el proveedor experimenta interrupciones. La transición a un nodo IPFS propio (por ejemplo, Kubo) eliminaría esta dependencia a costa de mayor complejidad operativa.

El segundo desafío es la variabilidad en los tiempos de minado de la red BlockDAG: si bien el promedio en las pruebas por escenarios se sitúa en $\approx 3,5$ s, los valores individuales pueden desviarse significativamente, especialmente bajo condiciones de congestión de la red pública de pruebas. En una red BlockDAG privada o permissionada, opción viable para despliegues municipales, este comportamiento sería más predecible y controlable.

El tercer desafío es la configuración de endpoints de la API: durante las primeras corridas del protocolo de prueba se registraron respuestas HTTP 307 (redirección temporal) provocadas por la ausencia de la barra final en las rutas de subida y descarga. Una vez corregida la configuración, todas las operaciones subsiguientes retornaron HTTP 200 con el ciclo completo de almacenamiento ejecutado. Este incidente ilustra la importancia de una validación exhaustiva del entorno antes de ejecutar pruebas formales, y la necesidad de contar con scripts de verificación previa del entorno como parte del proceso de despliegue.

Las evaluaciones cubren los dos módulos nucleares del sistema: traffic-sync demostró escalabilidad lineal ($\approx 0,034$ s/sensor) y operación estable en menos de 220 MB de RAM; traffic-storage alcanzó confirmación BlockDAG en ≈ 1 s a un costo inferior a $\$1,2 \times 10^{-5}$ USD por transacción, superando a Ethereum en latencia de confirmación ($>95\%$) y costo operativo (>4 órdenes de magnitud). Estos resultados establecen la base cuantitativa para las conclusiones del capítulo

siguiente.

CAPÍTULO 9

CONCLUSIÓN

El presente trabajo de tesis abordó el diseño, la implementación y la evaluación experimental de un sistema modular y descentralizado para la optimización del tráfico urbano y la persistencia verificable de datos asociados. La arquitectura propuesta integra simulación de tráfico mediante SUMO, clasificación de congestión con inferencia difusa tipo Mamdani, ajuste adaptativo de temporización semafórica mediante optimización por enjambre de partículas (PSO) y almacenamiento distribuido basado en IPFS y contratos inteligentes sobre una red BlockDAG. A diferencia de plataformas centralizadas convencionales, el sistema procesa datos provenientes de fuentes externas heterogéneas, expone sus capacidades mediante APIs REST y garantiza trazabilidad e integridad de los registros sin depender de un operador único de confianza.

Los capítulos anteriores desarrollaron los fundamentos teóricos, el diseño de cada módulo y su interacción dentro del flujo completo: desde la generación de escenarios controlados y la detección de cuellos de botella en traffic-sim, pasando por el análisis y la optimización en traffic-sync, hasta la publicación auditable de resultados en traffic-storage. El capítulo de evaluación experimental consolidó estas piezas con mediciones reproducibles en hardware de consumo, demostrando que el sistema es viable para entornos municipales con recursos limitados.

9.1. Resultados principales

La evaluación del módulo traffic-sync confirmó una escalabilidad aproximadamente lineal del tiempo de procesamiento conforme aumenta el número de sensores simulados: para 2000 sensores, el ciclo completo de análisis y optimización se completó en 67,6 s ($\approx 0,034$ s por sensor). El consumo de memoria RAM se mantuvo estable entre 198 y 219 MB en todos los escenarios, y el uso de CPU disminuyó del 72,3 % al 50,3 % a medida que creció la concurrencia, comportamiento coherente con el procesamiento asíncrono de FastAPI y con el bajo costo

computacional unitario de los componentes difuso y PSO. Estos resultados respaldan la viabilidad de despliegues sobre hardware embebido de bajo costo, como unidades Raspberry Pi en campo.

En traffic-storage, las transacciones de almacenamiento alcanzaron confirmación en la red BlockDAG en ≈ 1 s, con costos inferiores a $\$1,2 \times 10^{-5}$ USD por transacción. Los tiempos de subida extremo a extremo se situaron entre 6,7 y 7,3 s según la densidad vehicular del escenario, mientras que la descarga se completó en $\approx 1,75$ s, adecuado para flujos de auditoría y consulta sobre datos agregados de ciclos completados, no sobre control en tiempo estricto. La verificación de integridad mediante el CID de IPFS demostró que los contenidos recuperados conservan fielmente el original. Frente a Ethereum, el sistema superó en más del 95 % la latencia de confirmación y en más de cuatro órdenes de magnitud el costo operativo; frente a arquitecturas IoT-cloud centralizadas, la reducción de latencia de persistencia verificable superó el 60 %, con inmutabilidad garantizada por consenso distribuido y no por confianza en un operador central.

En conjunto, los experimentos establecen que la combinación IPFS+BlockDAG constituye una alternativa técnicamente competitiva para monitoreo urbano de alta frecuencia con requisitos de trazabilidad auditables, mientras que el núcleo analítico escala de forma predecible hacia escenarios de optimización por zonas urbanas.

9.2. Divulgación científica

Los hallazgos centrales de esta investigación fueron sintetizados y publicados en el artículo de conferencia internacional titulado *Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments*, presentado en la *2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies* (CHILECON), celebrada en Valparaíso, Chile, en octubre de 2025.¹ El artículo documenta, en formato condensado, la arquitectura modular del sistema, las métricas de escalabilidad y almacenamiento aquí ampliadas, y la comparación con plataformas basadas en Ethereum y soluciones centralizadas. La publicación valida externamente los resultados experimentales de la tesis y posiciona la propuesta en el ámbito de sistemas inteligentes para ciudades con almacenamiento verificable y despliegue en dispositivos de bajo costo.

9.3. Limitaciones

Pese a los resultados favorables, deben reconocerse limitaciones relevantes para un despliegue productivo. En primer lugar, las evaluaciones se realizaron principalmente en entorno simulado y sobre hardware de laboratorio; la transferencia a escenarios urbanos reales implica heterogeneidad de sensores, variabilidad en las comunicaciones y resistencia institucional a soluciones descentralizadas, aspectos aún no validados en pilotos de campo.

¹Disponible en IEEE Xplore: <https://ieeexplore.ieee.org/document/11476672>.

En segundo lugar, el módulo traffic-sync procesa actualmente un sensor por solicitud; si bien la arquitectura y las pruebas de estrés anticipan la evaluación conjunta por zonas, la optimización multivariable por clústeres de semáforos permanece como trabajo pendiente para lograr una dispersión efectiva del tráfico a escala territorial.

En tercer lugar, traffic-storage dependió de un nodo IPFS gestionado por terceros (Pinata), introduce variabilidad en los tiempos de minado de la red pública de pruebas BlockDAG y concentra el cuello de botella en la confirmación on-chain ($\approx 3,5$ s dentro del pipeline de subida). No se implementaron aún mecanismos activos de tolerancia a fallos ni replicación automática de contenido.

Finalmente, la configuración operativa de las APIs requiere validación exhaustiva del entorno antes del despliegue, como evidenció el incidente de redirecciones HTTP 307 durante las primeras corridas del protocolo de prueba.

9.4. Trabajo futuro

Como continuación natural de este trabajo, se identifican las siguientes líneas de investigación y desarrollo:

- Implementar agrupamiento jerárquico de sensores y optimización PSO por zonas urbanas, habilitando la coordinación de clústeres de semáforos en paralelo.
- Integrar fuentes de datos reales provenientes de dispositivos de conteo, sensores IoT con conectividad LoRaWAN y sistemas de visión por computadora.
- Desplegar nodos IPFS propios y evaluar redes BlockDAG privadas o permissionadas para despliegues municipales, reduciendo la variabilidad de confirmación y la dependencia de proveedores externos.
- Optimizar la gestión de nonces y la precarga del estado de gas para acortar la fase de confirmación en el pipeline de almacenamiento.
- Diseñar esquemas de comunicación asíncrona mediante instancias intermedias que alimenten en tiempo real los módulos del sistema desde el campo.
- Ejecutar pilotos en ciudades intermedias de la región para medir resiliencia, calidad de servicio e impacto en la fluidez del tráfico frente a plataformas centralizadas.
- Explorar la integración con marcos de *digital twins* urbanos para planificación y simulación avanzada.

9.5. Síntesis final

El sistema desarrollado en esta tesis ofrece una base tecnológica sólida que responde a criterios de trazabilidad, seguridad, integridad y eficiencia operativa en la gestión del tráfico urbano. Su enfoque modular y descentralizado, respaldado por evidencia cuantitativa y por la publicación derivada en CHILECON 2025, lo posiciona como una propuesta viable para municipios con presupuesto limitado. Las mejoras futuras deberán cerrar la brecha entre los resultados de laboratorio y la evidencia práctica en entornos urbanos reales, fortaleciendo la tolerancia a fallos y la optimización territorial del control semafórico.

REFERENCIAS

- [1] Markos Papageorgiou, Christina Diakaki, Vaya Dinopoulou, Apostolos Kotsialos y Yibing Wang. «Review of Road Traffic Control Strategies». En: *Proceedings of the IEEE* 91.12 (2003), págs. 2043-2067. DOI: 10.1109/JPROC.2003.819610.
- [2] Nicholas J. Garber y Lester A. Hoel. *Traffic and Highway Engineering*. 6th. Boston: Cengage Learning, 2019.
- [3] Aleksandar Stevanovic, Jelena Stevanovic, Kai Zhang y Stuart Batterman. «Optimizing Traffic Control to Reduce Fuel Consumption and Vehicular Emissions: Integrated Approach with VISSIM, CMEM, and VISGAOST». En: *Transportation Research Record* 2128 (2010). Incluye comparación de desempeño entre planes fijos y controladores adaptativos, págs. 105-113. DOI: 10.3141/2128-12.
- [4] Li Li, Yisheng Lv y Fei-Yue Wang. «Traffic Signal Timing via Deep Reinforcement Learning». En: *IEEE/CAA Journal of Automatica Sinica* 3.3 (2016). Deep Q-learning para control semafórico adaptativo, págs. 247-254. DOI: 10.1109/JAS.2016.7508798.
- [5] Michael Behrisch, Laura Bieker, Jakob Erdmann y Daniel Krajzewicz. «SUMO – Simulation of Urban MObility: An Overview». En: *Proceedings of the Third International Conference on Advances in System Simulation (SIMUL 2011)*. Barcelona, Spain: IARIA, 2011, págs. 63-68.
- [6] Lotfi A. Zadeh. «Fuzzy Sets». En: *Information and Control* 8.3 (1965), págs. 338-353. DOI: 10.1016/S0019-9958(65)90241-X.
- [7] James Kennedy y Russell Eberhart. «Particle Swarm Optimization». En: *Proceedings of the IEEE International Conference on Neural Networks*. Vol. 4. IEEE, 1995, págs. 1942-1948. DOI: 10.1109/ICNN.1995.488968.
- [8] Juan Benet. *IPFS - Content Addressed, Versioned, P2P File System*. Whitepaper fundacional de IPFS. 2014. arXiv: 1407.3561 [cs.DC]. URL: <https://arxiv.org/abs/1407.3561>.
- [9] Yonatan Sompolsky y Aviv Zohar. «PHANTOM and GHOSTDAG: A Scalable Generalization of Nakamoto Consensus». En: *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT '21)*. New York, NY, USA: ACM, 2021, págs. 57-70. DOI: 10.1145/3479722.3480990.

- [10] Kevin Mathias Galeano Saldivar y María José Duarte Kowalewski. «Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments». En: *2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON)*. Valparaíso, Chile: IEEE, 2025. DOI: 10.1109/CHILECON56137.2025.11476672.
- [11] Roger P. Roess, Elena S. Prassas y William R. McShane. *Traffic Engineering*. 5th. Upper Saddle River: Pearson, 2019.
- [12] Federal Highway Administration. *Traffic Signal Timing Manual*. <https://ops.fhwa.dot.gov/publications/fhwahop08024/>. FHWA-HOP-08-024, consultado: 2026-01-XX. 2008.
- [13] Ministerio de Obras Públicas y Comunicaciones. *Manual de Carreteras del Paraguay. Unidad 3: Diseño de Carreteras. Volumen 3.3: Señalización y Obras Complementarias*. Sección 3.3.2.7: Semáforos. Asunción, Paraguay: MOPC, 2019.
- [14] Nathan H. Gartner, John D. C. Little y Henry Gabbay. «Optimization of Traffic Signal Settings by Mixed-Integer Linear Programming: Part I: The Network Coordination Problem». En: *Transportation Science* 9.4 (1975). Formulación MILP para optimización de redes semafóricas, págs. 321-343. DOI: 10.1287/trsc.9.4.321.
- [15] S. C. Wong, H. Yang y H. K. Lo. «Traffic Signal Optimisation: Modeling and Computation». En: *Transportation Research Part B: Methodological* 32.3 (1998). Formulación matemática del problema de optimización de señales, págs. 157-172. DOI: 10.1016/S0191-2615(97)00028-X.
- [16] J. A. Hillier y R. Rothery. «The Synchronization of Traffic Signals for Minimum Delay». En: *Transportation Science* 1.2 (1967). Clásico sobre formulación de optimización de señales, págs. 81-94. DOI: 10.1287/trsc.1.2.81.
- [17] F. V. Webster. «Traffic Signal Settings». En: *Road Research Technical Paper* 39 (1958). Clásico método de diseño de tiempos semafóricos.
- [18] Byungkyu Park, Carroll J. Messer y Thomas Urbanik. «Traffic Signal Optimization Program for Oversaturated Conditions: Genetic Algorithm Approach». En: *Transportation Research Record* 1683 (1999). Uso de algoritmos genéticos para condiciones sobresaturadas, págs. 133-142. DOI: 10.3141/1683-17.
- [19] Halim Ceylan y Michael G. H. Bell. «Traffic Signal Timing Optimization Based on Genetic Algorithm Approach, Including Drivers' Routing». En: *Transportation Research Part B: Methodological* 38.4 (2004). Algoritmos genéticos para optimización semafórica, págs. 329-342. DOI: 10.1016/S0191-2615(03)00015-8.
- [20] Nathan H. Gartner, Farhad J. Pooran y Carleton M. Andrews. *Optimized Policies for Adaptive Control: A Case Study for Arterial Traffic*. Capítulos sobre control actuado y coordinado. Boston: Springer, 2001.

- [21] P. B. Hunt, D. I. Robertson, R. D. Bretherton y R. I. Winton. «SCOOT—A Traffic Responsive Method of Coordinating Signals». En: *Transport and Road Research Laboratory Report LR 1014* (1982). Sistema de control adaptativo SCOOT.
- [22] A. G. Sims y K. W. Dobinson. «The Sydney Coordinated Adaptive Traffic (SCATS) System: Philosophy and Benefits». En: *IEEE Transactions on Vehicular Technology* 29.2 (1980). Sistema SCATS de control adaptativo, págs. 130-137. DOI: 10.1109/T-VT.1980.23833.
- [23] X. Li et al. «A PSO Based Signal Timing Optimization Approach of Phase Combination». En: *Proceedings of the International Conference on Computer Technology and Engineering*. Aplicación de PSO a optimización de tiempos semafóricos. Atlantis Press, 2016, págs. 116-120.
- [24] R. Gonçalo et al. «On Tuning the Particle Swarm Optimization for Solving the Traffic Light Time Problem». En: *ICCSA 2022 – Lecture Notes in Computer Science*. PSO integrado con SUMO para optimización de semáforos. Springer, 2022.
- [25] Ebrahim H. Mamdani y Sedrak Assilian. «An Experiment in Linguistic Synthesis with a Fuzzy Logic Controller». En: *International Journal of Man-Machine Studies* 7.1 (1975), págs. 1-13. DOI: 10.1016/S0020-7373(75)80002-2.
- [26] Rizka Zuliani Musriroh. «Application of Mamdani Method on Fuzzy Logic to Decision of Traffic Light System». En: *Proceedings of International Conference on Informatics and Computational Sciences*. SciTePress, 2020, pág. 85200. DOI: 10.5220/0008520000000000.
- [27] Timothy J. Ross. *Fuzzy Logic with Engineering Applications*. 3rd. Chichester, UK: John Wiley & Sons, 2010.
- [28] Gökhan Erdiñç. «Application of a Mamdani-Based Fuzzy Traffic State Identifier». Tesis sobre identificación de estado de tráfico usando lógica difusa Mamdani. Thesis. Università "La Sapienza" di Roma, 2023. URL: https://iris.uniroma1.it/retrieve/d51daa72-738b-40a8-b0f8-2f25473ebf1b/Erdin%C3%A7_Application-Mamdani-based_2023.pdf.
- [29] Baher Abdulhai, Rob Pringle y Grigoris J. Karakoulas. «Reinforcement Learning for True Adaptive Traffic Signal Control». En: *Journal of Transportation Engineering* 129.3 (2003). Q-learning aplicado a control semafórico, págs. 278-285. DOI: 10.1061/(ASCE)0733-947X(2003)129:3(278).
- [30] Seung-Bae Cools, Carlos Gershenson y Bart D’Hooghe. «Self-Organizing Traffic Lights: A Realistic Simulation». En: *Advances in Applied Self-Organizing Systems*. Sistemas autoorganizados para control semafórico. Springer, 2013, págs. 45-55. DOI: 10.1007/978-1-4471-5113-5_3.

- [31] S. Erdogan, Bashar Al-Omari y Hesham Rakha. «Signal Timing Optimization for Coordinated Arterials: A Review of Progression-Based Methods». En: *Journal of Transportation Engineering* 139.4 (2013), págs. 358-370. DOI: 10.1061/(ASCE)TE.1943-5436.0000506.
- [32] Pradeep Kumar Singh, Roshan Singh, Sukumar Nandi y Sukumar Nandi. «Managing Smart Traffic Lights Using Fog Computing and Blockchain». En: *IEEE Internet of Things Journal* 7.7 (2020). Blockchain aplicado a gestión de semáforos inteligentes, págs. 6243-6252. DOI: 10.1109/JIOT.2020.2968410.
- [33] Ye Yuan y Fei-Yue Wang. «Blockchain and Cryptocurrencies: Model, Techniques, and Applications». En: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 48.9 (2018). Aplicaciones de blockchain en sistemas ciber-físicos, págs. 1421-1428. DOI: 10.1109/TSMC.2018.2854904.
- [34] Mauro Dell'Orco, Ozgur Baskan y Mario Marinelli. «A Harmony Search Algorithm Approach for Optimizing Traffic Signal Timings». En: *PROMET - Traffic & Transportation* 25.4 (2013). Metaheurísticas aplicadas a optimización semaforica, págs. 349-358. DOI: 10.7307/ptt.v25i4.1245.
- [35] Vahid Azimirad, Naser Pariz y Saeed Balochian. «A Novel Fuzzy Model and Control of Single Intersection at Urban Traffic Network». En: *IEEE Systems Journal* 4.1 (2010). Modelo difuso para intersecciones urbanas, págs. 107-111. DOI: 10.1109/JSYST.2010.2043159.
- [36] Sunil Kumar Yadav y Richa Sharma. «Fuzzy Logic Based Traffic Light Controller for Heterogeneous Traffic». En: *International Journal of Advanced Research in Computer Science* 12.3 (2021). Control difuso para tráfico heterogéneo, págs. 1-5.
- [37] Nazri Mohd Nawi, Azlan Khan y M. Z. Rehman. «A New Levenberg-Marquardt based Back Propagation Algorithm Trained with Cuckoo Search». En: *Procedia Technology* 11 (2013). Optimización de sistemas neuro-fuzzy, págs. 18-23.
- [38] DLR Institute of Transportation Systems. *SUMO User Documentation*. Consultado: 2025-12-XX. Eclipse SUMO. 2025. URL: <https://sumo.dlr.de/docs/>.
- [39] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. Consultado: 2025-01-XX. 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
- [40] Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller y Steven Goldfeder. *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. Princeton University Press, 2016. ISBN: 978-0-691-17169-2.
- [41] Vitalik Buterin. «Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform». En: (2014). Whitepaper, Consultado: 2025-01-XX. URL: <https://ethereum.org/en/whitepaper/>.

- [42] Juan Garay, Aggelos Kiayias y Nikos Leonardos. «The Bitcoin Backbone Protocol: Analysis and Applications». En: *Proceedings of the 19th International Conference on Principles of Distributed Systems (OPODIS 2015)*. Consultado: 2025-01-XX. 2015, págs. 1-15. URL: https://opodis2015.irisa.fr/files/2016/01/Garay_OPODIS_2015.pdf.
- [43] Qiang Wang, Jiajing Yu, Shuo Chen y Yong Xiang. «SoK: DAG-based Blockchain Systems». En: *ACM Computing Surveys* 55.12 (2023), Article 256. DOI: 10.1145/3576899.
- [44] X. Zhang et al. «SoK: DAG-based Consensus Protocols». En: *arXiv preprint* (2025). Systematization of Knowledge sobre protocolos de consenso basados en DAG. arXiv: 2411.10026 [cs.CR]. URL: <https://arxiv.org/abs/2411.10026>.
- [45] Ethereum Foundation. *Blocks*. Documentación oficial de Ethereum sobre block time de 12 segundos en PoS. 2024. URL: <https://ethereum.org/developers/docs/blocks/>.
- [46] IPFS Project. *IPFS Architecture*. Especificación oficial de arquitectura IPFS, define formalmente $\text{nodeID} := \text{multihash}(\text{publicKey})$ y esquema de identidad basado en PKI. 2024. URL: <https://github.com/ipfs/specs/blob/main/ARCHITECTURE.md>.
- [47] Lotfi A. Zadeh. «Similarity Relations and Fuzzy Orderings». En: *Information Sciences* 3.2 (1971), págs. 177-200. DOI: 10.1016/S0020-0255(71)80005-1.
- [48] Segismundo S. Izquierdo y Luis R. Izquierdo. «Mamdani Fuzzy Systems for Modelling and Simulation». En: *Journal of Artificial Societies and Social Simulation* 21.3 (2018), pág. 2. URL: <https://www.jasss.org/21/3/2.html>.
- [49] Russell C. Eberhart y James Kennedy. «A New Optimizer Using Particle Swarm Theory». En: *Proceedings of the Sixth International Symposium on Micro Machine and Human Science (MHS '95)*. IEEE, 1995, págs. 39-43. DOI: 10.1109/MHS.1995.494215.
- [50] Maurice Clerc y James Kennedy. «The Particle Swarm – Explosion, Stability, and Convergence in a Multidimensional Complex Space». En: *IEEE Transactions on Evolutionary Computation* 6.1 (2002), págs. 58-73. DOI: 10.1109/4235.985692.
- [51] Yuhui Shi y Russell Eberhart. «Parameter Selection in Particle Swarm Optimization». En: (1998). Proposed inertia weight adaptation, págs. 591-600.
- [52] Riccardo Poli, James Kennedy y Tim Blackwell. «Particle Swarm Optimization: An Overview». En: *Swarm Intelligence* 1.1 (2007), págs. 33-57. DOI: 10.1007/s11721-007-0002-0.
- [53] Marco Conoscenti, Antonio Vetro y Juan Carlos De Martin. «Blockchain for the Internet of Things: A Survey». En: *2016 IEEE 19th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. IEEE, 2016, págs. 1-6. DOI: 10.1109/SNPD.2016.7515925.

- [54] Florian Eckermann, Hendrik Beckmann, Benjamin Hagedorn y Marcus Kaulartz. «Blockchain Technology in the Energy Sector: A Systematic Review of Challenges and Opportunities». En: *Renewable and Sustainable Energy Reviews* 100 (2019). Análisis de costos y limitaciones de blockchain, págs. 143-174. DOI: 10.1016/j.rser.2018.10.014.